

Aula 07

*DataPrev (Perfil 3: Desenvolvimento de Software) Engenharia de Software - 2026
(Pós-Edital)*

Autor:

**Diego Carvalho, Equipe
Informática 2 (Diego Carvalho),
Equipe Informática 1 (Diego
Carvalho)**

09 de Julho de 2026

Índice

1) Arquitetura Web - Teoria	3
2) Arquitetura Web - Questões Comentadas - FGV	32
3) Arquitetura Web - Lista de Questões - FGV	41

ARQUITETURA WEB

Conceitos Básicos

ARQUITETURA WEB

Arquitetura Web é o conjunto de princípios, padrões, tecnologias e componentes utilizados para estruturar aplicações que operam sobre a Web. Ela define como clientes, servidores, bancos de dados, APIs e demais elementos interagem para fornecer funcionalidades aos usuários. Seu objetivo é garantir organização, escalabilidade, desempenho, segurança e facilidade de manutenção, servindo como base para o desenvolvimento de sistemas modernos acessados por navegadores, aplicativos móveis e outros clientes conectados à Internet.

A Arquitetura Web corresponde ao **conjunto de princípios, padrões, tecnologias e decisões estruturais utilizados para projetar e organizar aplicações executadas na Web**. Ela define como os componentes do sistema se comunicam, como os dados são processados e armazenados e quais mecanismos serão utilizados para atender requisitos de desempenho, segurança, escalabilidade, disponibilidade e manutenção. Por que tantas decisões precisam ser tomadas antes mesmo da implementação? Porque **a estrutura escolhida influencia diretamente a capacidade de evolução e operação do sistema**.

Quando se fala em arquitetura web, o foco não está apenas no código ou nas interfaces visíveis ao usuário. A estrutura envolve navegadores, servidores, bancos de dados, APIs, protocolos de comunicação, mecanismos de autenticação, infraestrutura de hospedagem e integrações com serviços externos. Surge então uma dúvida comum: por que considerar tantos elementos ao mesmo tempo? Porque todos participam da execução da aplicação e contribuem para o funcionamento correto do ambiente como um todo.

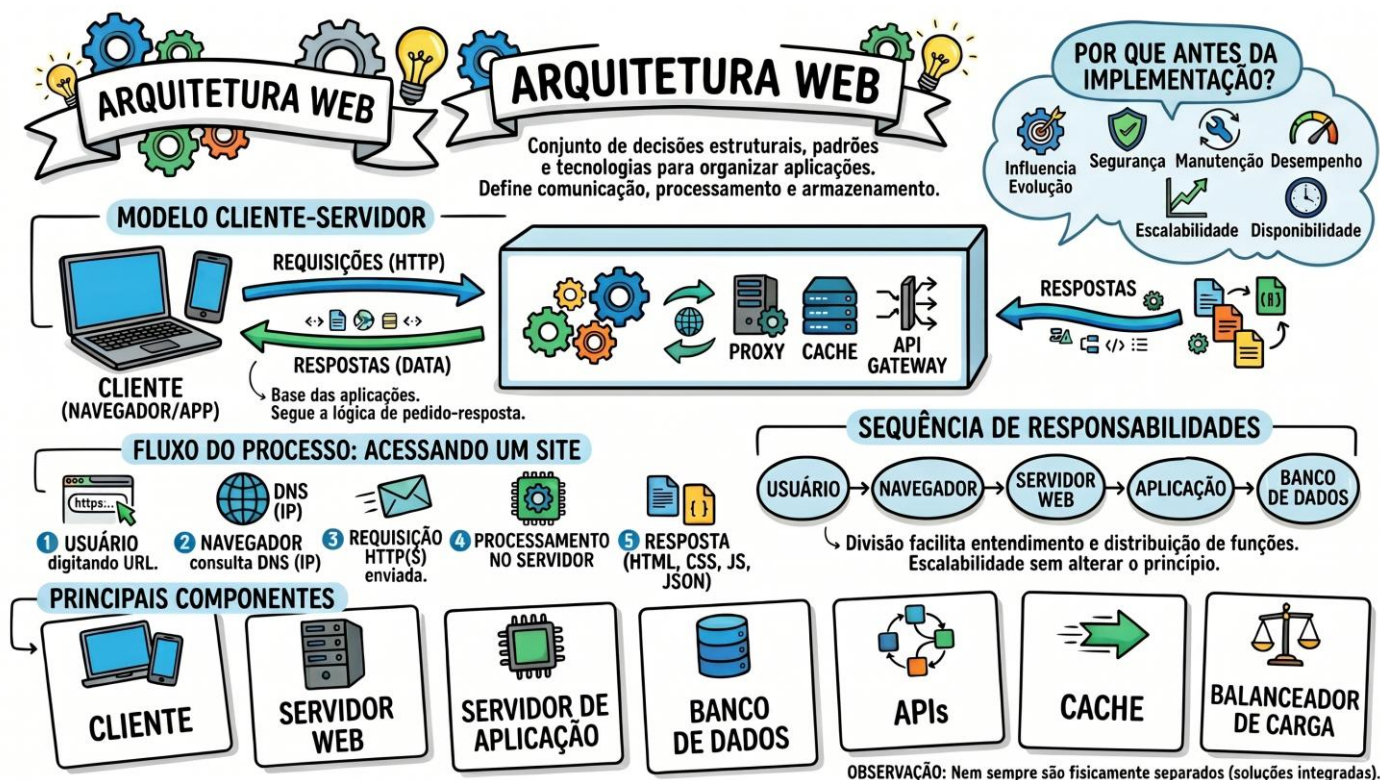
A base da maioria das aplicações web continua sendo o modelo cliente-servidor. Nesse modelo, **um cliente, normalmente um navegador ou aplicativo móvel, envia requisições para um servidor**. O servidor recebe essas solicitações, executa os processamentos necessários e devolve uma resposta apropriada. Embora arquiteturas modernas incluam componentes intermediários, como proxies, caches distribuídos e gateways de API, **a comunicação fundamental continua seguindo essa lógica de solicitação e resposta**.

O funcionamento desse processo pode ser observado durante o acesso a um site. O usuário informa uma URL e o sistema realiza consultas ao serviço DNS para descobrir o endereço IP associado ao domínio. Em seguida, uma requisição HTTP ou HTTPS é enviada ao servidor. O que acontece depois que essa solicitação chega ao destino? O servidor executa os processamentos necessários, consulta recursos adicionais quando necessário e retorna conteúdos como páginas HTML, arquivos CSS, scripts JavaScript ou dados estruturados em formatos como JSON.

Uma visão simplificada desse fluxo pode ser representada pela sequência formada pelo usuário, navegador, servidor web, aplicação e banco de dados. Cada elemento possui uma responsabilidade específica dentro da comunicação. Essa divisão facilita o entendimento da arquitetura e permite distribuir funções de maneira mais organizada. Conforme a aplicação cresce, novos componentes podem ser adicionados sem alterar o princípio básico de interação entre cliente e servidor.

Entre os principais componentes de uma arquitetura web estão o cliente, responsável pela interação com o usuário; o servidor web, que recebe e direciona requisições; o servidor de aplicação, que executa regras de negócio; o banco de dados, que armazena informações; as APIs, que viabilizam integrações; **o cache**,

que reduz acessos repetitivos a recursos mais lentos; e o balanceador de carga, que distribui requisições entre múltiplos servidores. Seria obrigatório que todos esses componentes existissem separadamente? Nem sempre, pois algumas soluções concentram várias dessas funções em uma única plataforma ou serviço.



Objetivos da Arquitetura

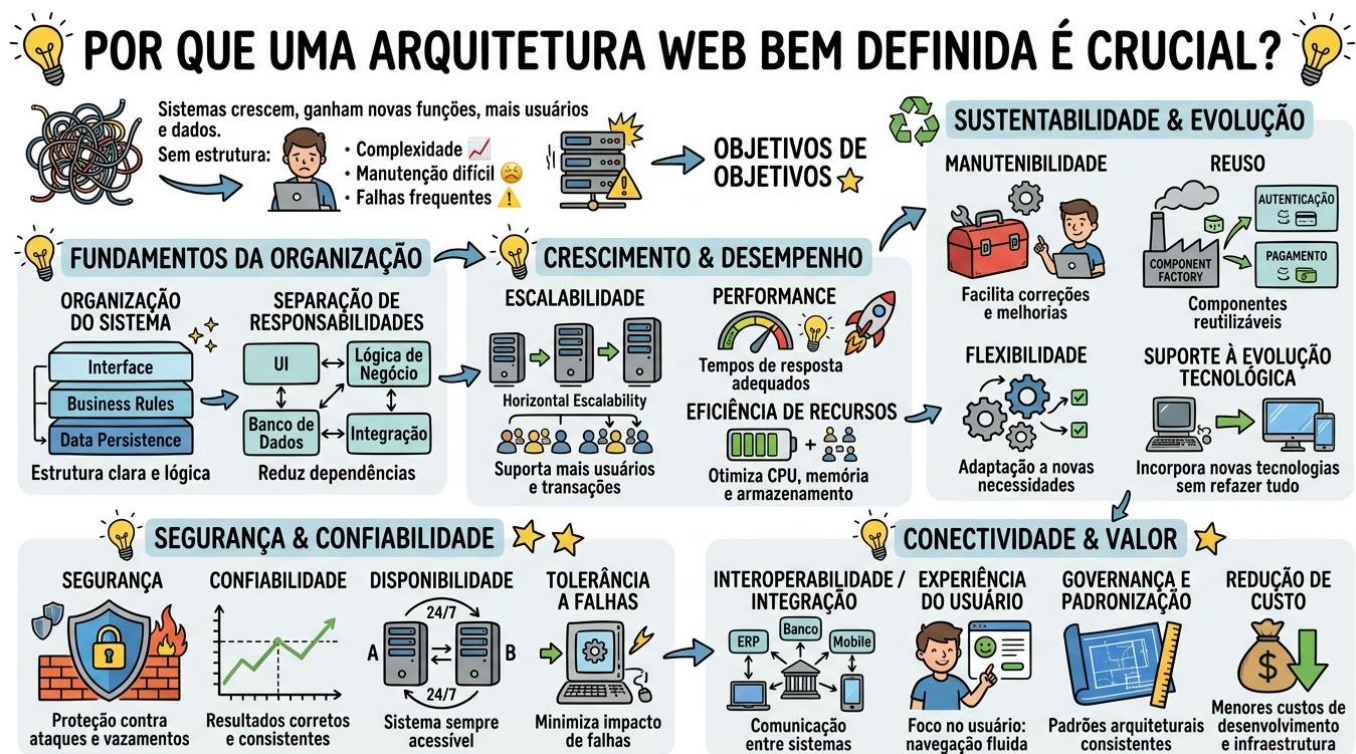
INCIDÊNCIA EM PROVA: BAIXÍSSIMA

Uma arquitetura web adequada tem como objetivo estruturar o sistema de forma organizada, permitindo que ele atenda às necessidades atuais e suporte sua evolução ao longo do tempo. Para isso, define responsabilidades claras para cada componente, reduz o acoplamento e favorece a reutilização de funcionalidades. Além disso, busca garantir escalabilidade para lidar com o crescimento de usuários, dados e transações, bem como manutenibilidade para facilitar correções, atualizações e implementação de novas funcionalidades com menor esforço e risco.

Outro objetivo fundamental é assegurar que a aplicação possa se integrar adequadamente a sistemas externos, como APIs, bancos, ERPs e serviços de terceiros. A arquitetura também deve incorporar mecanismos de segurança para proteger dados e recursos contra ameaças e acessos indevidos. Por fim, precisa promover alta disponibilidade, utilizando recursos como redundância, balanceamento de carga e monitoramento, de modo a manter o sistema acessível e operacional mesmo diante de falhas ou situações adversas.

OBJETIVOS	DESCRIÇÃO
ORGANIZAÇÃO DO SISTEMA	Estruturar a aplicação em componentes e camadas bem definidas, facilitando o entendimento, desenvolvimento e manutenção do software.
SEPARAÇÃO DE RESPONSABILIDADES	Distribuir funções entre diferentes módulos (interface, regras de negócio, persistência, integração etc.), reduzindo dependências e aumentando a qualidade do código.
ESCALABILIDADE	Permitir que a aplicação suporte crescimento no número de usuários, transações e volume de dados sem perda significativa de desempenho.

OBJETIVOS	DESCRIÇÃO
DESEMPENHO	Garantir tempos de resposta adequados por meio da otimização de recursos, uso de cache, balanceamento de carga e outras estratégias arquiteturas.
MANUTENIBILIDADE	Facilitar correções, melhorias e evolução do sistema ao longo do tempo, reduzindo o custo de manutenção.
REUTILIZAÇÃO	Possibilitar que componentes, serviços e funcionalidades sejam reutilizados em diferentes partes da aplicação ou em outros sistemas.
SEGURANÇA	Proteger dados, usuários e recursos contra acessos não autorizados, ataques e vazamentos de informações.
DISPONIBILIDADE	Garantir que o sistema permaneça acessível e operacional pelo maior tempo possível, mesmo diante de falhas de hardware ou software.
CONFIABILIDADE	Assegurar que o sistema execute suas funções corretamente e produza resultados consistentes e previsíveis.
INTEROPERABILIDADE	Permitir a integração e comunicação entre sistemas, plataformas, dispositivos e tecnologias diferentes.
FLEXIBILIDADE	Facilitar adaptações a novas necessidades de negócio, tecnologias emergentes e mudanças de requisitos.
ESCALABILIDADE HORIZONTAL	Possibilitar o aumento da capacidade do sistema por meio da adição de novos servidores ou instâncias.
TOLERÂNCIA A FALHAS	Minimizar impactos de falhas por meio de redundância, replicação e mecanismos de recuperação automática.
EFICIÊNCIA NO USO DE RECURSOS	Utilizar processamento, memória, armazenamento e rede de forma otimizada para reduzir custos operacionais.
EXPERIÊNCIA DO USUÁRIO	Proporcionar navegação fluida, respostas rápidas, disponibilidade contínua e interfaces adequadas às necessidades dos usuários.
SUSTENTAÇÃO DA EVOLUÇÃO TECNOLÓGICA	Permitir a incorporação gradual de novas tecnologias, frameworks e padrões sem exigir a reconstrução completa do sistema.
GOVERNANÇA E PADRONIZAÇÃO	Estabelecer padrões arquiteturas que orientem equipes de desenvolvimento, promovendo consistência e qualidade nas soluções.
REDUÇÃO DE CUSTOS	Diminuir custos de desenvolvimento, operação, manutenção e infraestrutura por meio de uma arquitetura eficiente e bem planejada.



Qualidades Arquiteturais

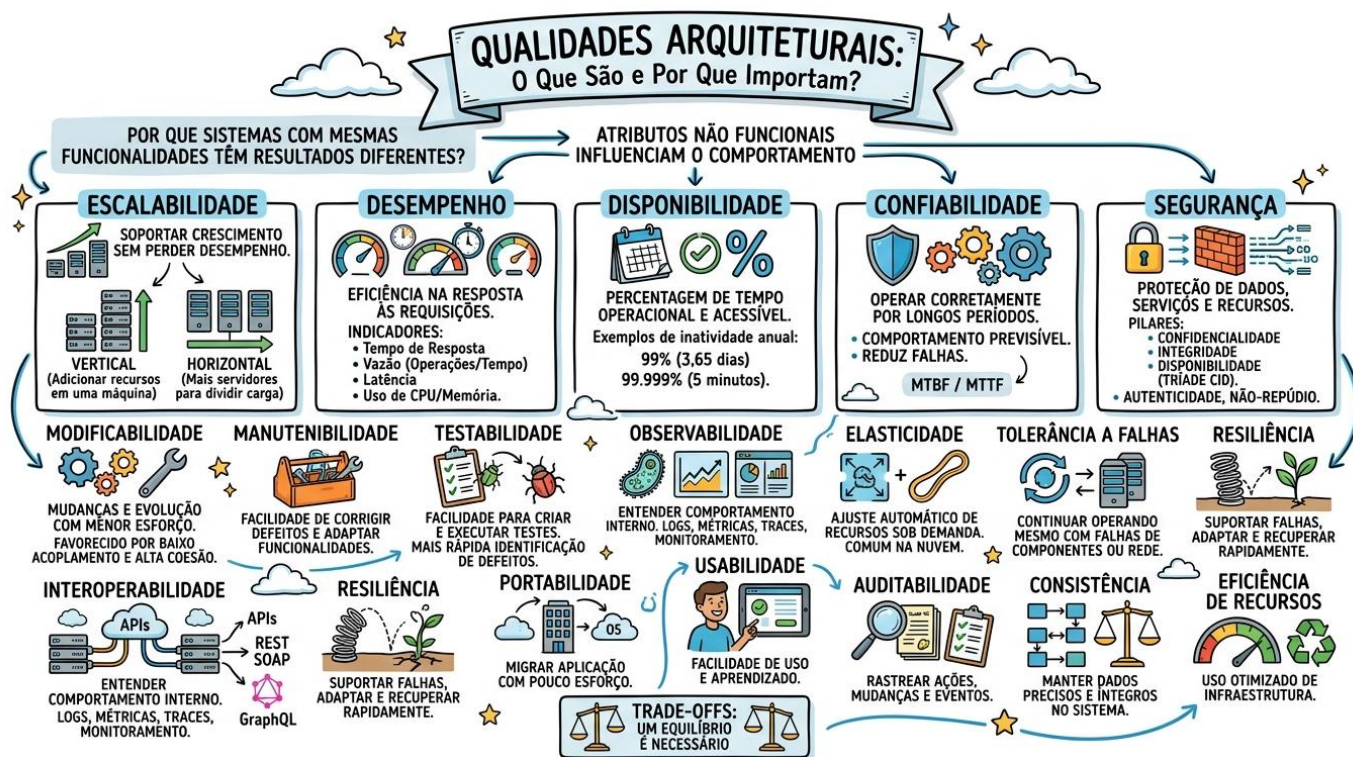
INCIDÊNCIA EM PROVA: BAIXÍSSIMA

As qualidades arquiteturais, também chamadas de atributos de qualidade, são características não funcionais que influenciam diretamente o comportamento e a eficiência de um sistema. Entre as mais importantes estão a escalabilidade, que permite suportar crescimento de usuários e dados; o desempenho, relacionado à rapidez e eficiência das respostas; a disponibilidade, que mede o tempo em que o sistema permanece operacional; a confiabilidade, associada à estabilidade e previsibilidade; e a segurança, responsável por proteger informações e recursos contra acessos indevidos e outras ameaças.

Outros atributos relevantes incluem a modificabilidade, que facilita alterações e evolução do software; a interoperabilidade, que possibilita integração com outros sistemas; a testabilidade, que favorece a criação e execução de testes; e a observabilidade, que permite compreender o comportamento interno da aplicação por meio de logs, métricas e monitoramento. Em conjunto, esses atributos ajudam a garantir que o sistema permaneça eficiente, seguro, confiável e adaptável às mudanças de negócio e às demandas futuras.

QUALIDADE ARQUITETURAL	DESCRIÇÃO
ESCALABILIDADE	Capacidade de um sistema aumentar sua capacidade de processamento, armazenamento ou atendimento de usuários à medida que a demanda cresce. Pode ocorrer por ampliação dos recursos de uma máquina ou pela adição de novos servidores.
DESEMPENHO	Refere-se à eficiência com que o sistema executa suas funções e responde às solicitações. Envolve aspectos como tempo de resposta, latência, throughput e utilização adequada dos recursos computacionais disponíveis.
DISPONIBILIDADE	Representa o percentual de tempo em que o sistema permanece operacional e acessível aos usuários. Quanto maior a disponibilidade, menor a ocorrência de interrupções que possam afetar a continuidade dos serviços prestados.

QUALIDADE ARQUITETURAL	DESCRIÇÃO
CONFIABILIDADE	Capacidade de o sistema executar suas funções corretamente durante longos períodos, produzindo resultados previsíveis e consistentes. Busca minimizar falhas inesperadas e garantir estabilidade operacional contínua.
SEGURANÇA	Característica voltada à proteção de dados, serviços e recursos contra acessos indevidos, alterações não autorizadas e ataques. Abrange mecanismos de autenticação, autorização, criptografia e auditoria de eventos.
MODIFICABILIDADE	Capacidade de realizar alterações, correções ou evoluções no sistema com esforço reduzido e impacto limitado sobre outros componentes. É favorecida por modularização adequada, alta coesão e baixo acoplamento.
INTEROPERABILIDADE	Capacidade de diferentes sistemas, plataformas ou aplicações trocarem informações e colaborarem entre si de forma eficiente. Depende da adoção de padrões, protocolos e contratos de comunicação bem definidos.
TESTABILIDADE	Refere-se à facilidade de criar, executar e manter testes durante o desenvolvimento e a manutenção do software. Sistemas testáveis permitem identificar defeitos rapidamente e validar alterações com menor esforço.
OBSERVABILIDADE	Capacidade de compreender o comportamento interno do sistema por meio da análise de logs, métricas, traces e monitoramento. Facilita diagnósticos, identificação de falhas e análise de desempenho operacional.
MANUTENIBILIDADE	Característica que expressa a facilidade de corrigir defeitos, adaptar funcionalidades e evoluir o software ao longo do tempo. Sistemas bem estruturados tendem a apresentar menor custo e esforço de manutenção.
USABILIDADE	Mede o grau de facilidade com que usuários conseguem aprender, compreender e utilizar o sistema para atingir seus objetivos. Interfaces intuitivas e consistentes contribuem diretamente para essa característica.
PORTABILIDADE	Capacidade de executar ou migrar uma aplicação entre diferentes ambientes, sistemas operacionais, plataformas ou provedores com pouco esforço. Reduz dependências tecnológicas excessivas e amplia a flexibilidade.
REUSABILIDADE	Capacidade de reutilizar componentes, módulos, bibliotecas ou serviços em diferentes projetos ou partes da aplicação. Favorece padronização, redução de retrabalho e aumento da produtividade das equipes.
ELASTICIDADE	Capacidade de aumentar ou reduzir automaticamente os recursos computacionais disponíveis conforme as variações de demanda. É uma característica amplamente associada a ambientes de computação em nuvem.
TOLERÂNCIA A FALHAS	Capacidade de continuar operando adequadamente mesmo quando ocorrem falhas em componentes, servidores ou conexões de rede. Normalmente depende de mecanismos de redundância e recuperação automática.
RESILIÊNCIA	Capacidade de resistir a falhas, adaptar-se a situações adversas e recuperar rapidamente sua operação normal. Vai além da tolerância a falhas ao enfatizar a recuperação e continuidade do negócio.
AUDITABILIDADE	Capacidade de registrar e rastrear ações, alterações e eventos relevantes ocorridos no sistema. Permite investigações, conformidade regulatória, identificação de responsabilidades e reconstrução de ocorrências.
CONSISTÊNCIA	Capacidade de manter os dados corretos, íntegros e coerentes ao longo do tempo, mesmo em ambientes distribuídos. Garante que diferentes componentes possuam uma visão adequada das informações.
EFICIÊNCIA DE RECURSOS	Capacidade de utilizar processamento, memória, armazenamento e rede de forma otimizada, entregando bons resultados com menor consumo de recursos e melhor aproveitamento da infraestrutura disponível.



Evolução de Arquiteturas Web

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

A evolução da arquitetura web acompanhou o crescimento da Internet e das necessidades das organizações. Inicialmente predominavam os sites estáticos, compostos por páginas HTML fixas armazenadas no servidor. Com o aumento da interação dos usuários, surgiram as aplicações dinâmicas baseadas em tecnologias como PHP, ASP e JSP, capazes de gerar conteúdo em tempo real a partir de bancos de dados. Posteriormente, a arquitetura em camadas consolidou a separação entre apresentação, lógica de negócio e dados, enquanto o SOA promoveu a reutilização de funcionalidades por meio de serviços compartilhados entre diferentes aplicações.

Com a expansão das integrações, as APIs REST tornaram-se amplamente utilizadas graças à simplicidade do HTTP e do JSON. Em seguida, os microsserviços passaram a dividir sistemas em serviços independentes e especializados, favorecendo escalabilidade e implantação autônoma. A consolidação da computação em nuvem impulsionou o modelo Cloud Native, baseado em containers, automação e elasticidade. Mais recentemente, arquiteturas orientadas a eventos e soluções serverless ganharam destaque ao permitir processamento assíncrono e abstração da infraestrutura, tornando as aplicações mais flexíveis, escaláveis e adequadas aos ambientes modernos de nuvem.

ETAPAS	DESCRIÇÃO
SITES ESTÁTICOS	Predominantes nos anos 1990, eram compostos por páginas HTML armazenadas diretamente no servidor. O conteúdo era praticamente fixo e cada solicitação resultava apenas na entrega de arquivos previamente criados, sem processamento significativo.
APLICAÇÕES DINÂMICAS	Introduziram processamento no servidor por meio de tecnologias como PHP, ASP, JSP e CGI. As páginas passaram a ser geradas em tempo real com base em dados armazenados em bancos de dados e regras de negócio executadas durante cada requisição.
ARQUITETURA EM CAMADAS (THREE-TIER)	Organizou os sistemas em camadas de apresentação, negócio e dados. Essa separação permitiu distribuir responsabilidades de forma mais clara, facilitando manutenção, reutilização de componentes e evolução das aplicações corporativas.

ETAPAS	DESCRIÇÃO
SOA (SERVICE-ORIENTED ARCHITECTURE)	Passou a estruturar funcionalidades como serviços reutilizáveis acessíveis por diferentes sistemas. Essa abordagem favoreceu integração corporativa, redução de redundâncias e compartilhamento de capacidades de negócio entre aplicações distintas.
APIS REST	Popularizaram integrações mais simples e leves utilizando HTTP e formatos como JSON. Tornaram-se fundamentais para comunicação entre sistemas web, aplicativos móveis e serviços distribuídos, ampliando a interoperabilidade entre plataformas.
MICROSSERVIÇOS	Propõem dividir a aplicação em serviços pequenos, independentes e especializados. Cada serviço pode ser desenvolvido, implantado e escalado separadamente, aumentando flexibilidade, autonomia das equipes e capacidade de evolução contínua.
CLOUD NATIVE	Representa aplicações projetadas especificamente para ambientes de nuvem. Utiliza conceitos como containers, orquestração, automação, elasticidade e alta disponibilidade, explorando plenamente os recursos oferecidos pelos provedores de nuvem.
SERVERLESS	Modelo no qual a infraestrutura é administrada pelo provedor de nuvem. O desenvolvedor concentra-se só na implementação das funções de negócio, enquanto escalabilidade, provisionamento e gerenciamento dos servidores são automatizados.
EVENT-DRIVEN ARCHITECTURE (EDA)	Baseia-se na produção e consumo de eventos para coordenar componentes distribuídos. O processamento ocorre de forma assíncrona, permitindo maior desacoplamento, escalabilidade e capacidade de reação a acontecimentos do ambiente.



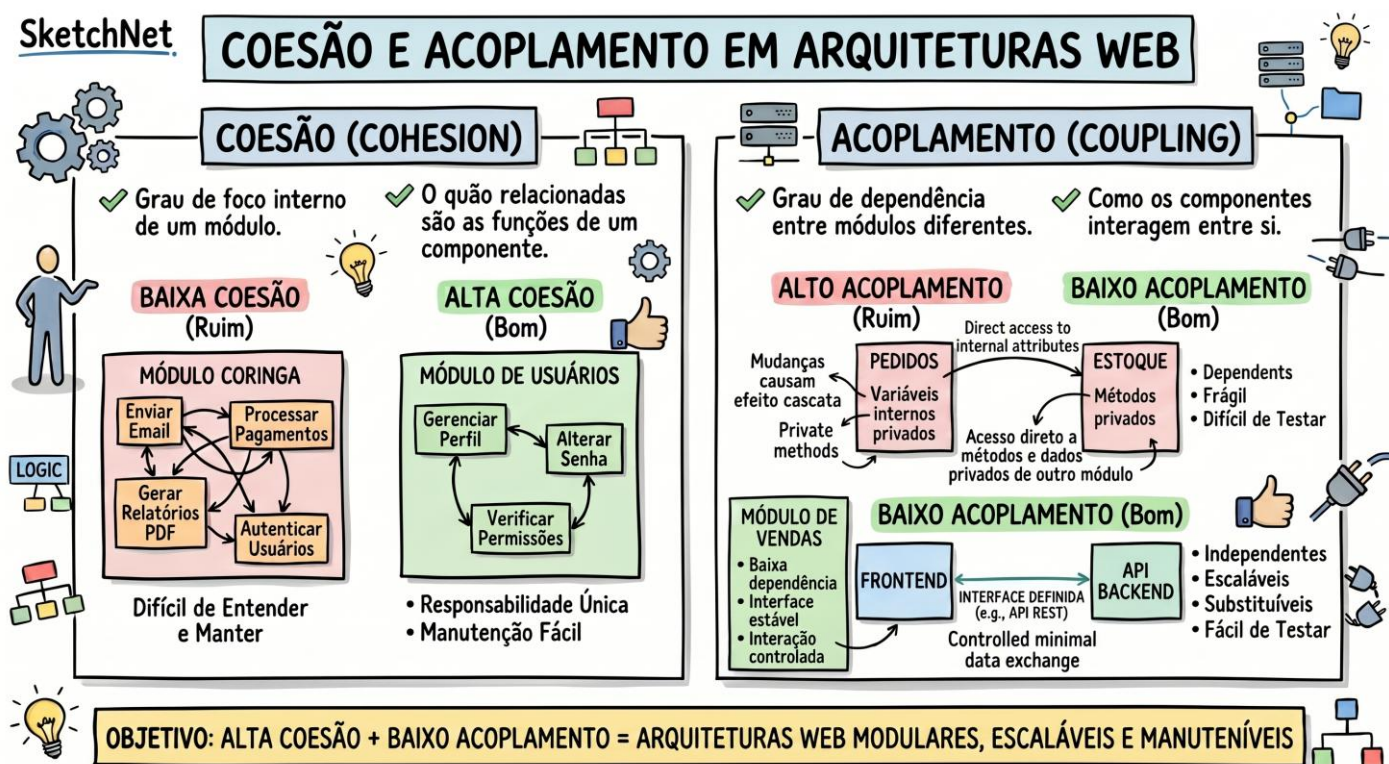
Coesão e Acoplamento

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

COESÃO E ACOPLAMENTO

Coesão e acoplamento são dois conceitos fundamentais da engenharia de software utilizados para avaliar a qualidade estrutural de um sistema. A coesão mede o grau de relacionamento entre as responsabilidades de um módulo, sendo desejável que cada componente execute uma função bem definida e específica (alta coesão). Já o acoplamento mede o nível de dependência entre módulos distintos, sendo preferível que essa dependência seja mínima (baixo acoplamento). Sistemas com alta coesão e baixo acoplamento tendem a ser mais fáceis de manter, testar, reutilizar e evoluir.

Coesão e acoplamento são conceitos fundamentais da Engenharia e da Arquitetura de Software. O acoplamento mede o grau de dependência entre módulos ou componentes: quanto menor essa dependência, mais fácil é modificar uma parte do sistema sem provocar impactos em outras. Por isso, arquiteturas com baixo acoplamento tendem a ser mais flexíveis, reutilizáveis e simples de manter ao longo do tempo.



A coesão, por sua vez, avalia o quanto as responsabilidades internas de um módulo estão relacionadas entre si. Componentes com alta coesão concentram funcionalidades voltadas para um objetivo específico, tornando seu comportamento mais fácil de compreender, testar e evoluir. Dessa forma, uma boa arquitetura busca combinar alta coesão e baixo acoplamento, produzindo sistemas mais organizados, manuteníveis e adaptáveis às mudanças de negócio e tecnologia. Vejamos na tabela abaixo os vários tipos de acoplamento:

TIPO DE ACOPLAMENTO	DESCRIÇÃO
ACOPLAMENTO POR CONTEÚDO	Ocorre quando um módulo acessa diretamente dados internos, estruturas ou elementos de controle pertencentes a outro módulo. É considerado o tipo mais forte e indesejável de acoplamento, pois cria elevada dependência entre componentes.
ACOPLAMENTO COMUM	Surge quando diversos módulos compartilham uma mesma área global de dados. Alterações na estrutura ou no conteúdo dessas informações podem impactar múltiplos componentes, dificultando manutenção, testes e controle de dependências.
ACOPLAMENTO POR CONTROLE	Ocorre quando um módulo envia informações que determinam o fluxo de execução de outro módulo. Nesse cenário, o componente chamado passa a depender de decisões externas para definir parte de seu comportamento interno.
ACOPLAMENTO POR DADOS	Caracteriza-se pela troca de dados simples entre módulos por meio de parâmetros necessários ao processamento. É considerado um dos acoplamentos mais desejáveis, pois limita a dependência ao compartilhamento explícito de informações.

ACOPLAMENTO POR CHAMADAS DE ROTINAS	Ocorre quando um módulo invoca procedimentos, funções ou métodos de outro módulo. Embora aumente a dependência entre componentes, é amplamente utilizado e frequentemente necessário para viabilizar a colaboração entre partes do sistema.
ACOPLAMENTO POR USO DE TIPOS	Surge quando um componente utiliza tipos de dados, classes ou estruturas definidos por outro componente. Mudanças nessas definições podem exigir modificações em todos os módulos que dependem desses tipos compartilhados.
ACOPLAMENTO POR INCLUSÃO OU IMPORTAÇÃO	Ocorre quando um componente incorpora bibliotecas, pacotes, módulos ou arquivos definidos por outro componente. Essa dependência estabelece um vínculo estrutural que pode propagar impactos decorrentes de alterações futuras.
ACOPLAMENTO EXTERNO	Acontece quando componentes dependem de recursos externos, como bancos de dados, sistemas operacionais, APIs, serviços de terceiros ou infraestrutura. Embora necessário, recomenda-se restringi-lo para reduzir impactos e riscos.

A coesão representa o grau de relacionamento entre atributos e métodos existentes dentro de uma mesma classe ou componente. Uma classe altamente coesa concentra responsabilidades relacionadas a um objetivo específico, tornando o sistema mais organizado, compreensível e fácil de manter. Em sistemas orientados a objetos, essa característica contribui para uma melhor divisão de responsabilidades e para a criação de componentes mais especializados.

O acoplamento, por sua vez, mede o nível de dependência entre classes ou componentes. Como a colaboração entre objetos é essencial nesse paradigma, algum grau de acoplamento é inevitável, mas dependências excessivas podem dificultar desenvolvimento, testes e manutenção. Por isso, uma das principais recomendações da Engenharia de Software é buscar alta coesão e baixo acoplamento, criando sistemas mais flexíveis, testáveis e preparados para evoluir com menor impacto diante de mudanças futuras.



Modelo Cliente/Servidor

MODELO CLIENTE/SERVIDOR

O modelo cliente/servidor é um paradigma de arquitetura de sistemas distribuídos no qual as responsabilidades são divididas entre clientes, que solicitam serviços, e servidores, que processam essas solicitações e fornecem respostas. O cliente normalmente é responsável pela interface com o usuário, enquanto o servidor concentra regras de negócio, processamento e acesso a dados. A comunicação ocorre por meio de uma rede utilizando protocolos específicos, como HTTP ou HTTPS. Esse modelo constitui a base da maioria das aplicações web modernas, permitindo centralização de recursos, compartilhamento de serviços e escalabilidade.

O modelo cliente-servidor é a base da maioria das aplicações web e divide o sistema em dois papéis principais. O cliente, normalmente um navegador ou aplicativo, interage com o usuário e envia solicitações pela rede, enquanto o servidor recebe essas requisições, processa os dados necessários e retorna respostas. Esse mecanismo de requisição e resposta ocorre continuamente durante a navegação, permitindo acesso a páginas, autenticação de usuários e execução de diversas funcionalidades.

Entre as principais vantagens desse modelo estão a centralização do processamento, o compartilhamento de recursos e a facilidade de manutenção. Como as regras de negócio e os dados permanecem concentrados nos servidores, atualizações podem ser realizadas sem modificar cada cliente individualmente. Além disso, a arquitetura favorece a segurança, ao manter informações críticas no servidor, e a escalabilidade, permitindo ampliar a capacidade do sistema por meio da adição de novos recursos e servidores conforme a demanda cresce.

ENTENDENDO O MODELO CLIENTE/SERVIDOR

ARQUITETURA DE SISTEMAS DISTRIBUÍDOS: DIVISÃO DE RESPONSABILIDADES



Cliente

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

CLIENTE

O cliente é o componente responsável por interagir diretamente com o usuário e solicitar serviços a um servidor. Sua função principal consiste em capturar ações do usuário, enviar requisições, receber respostas e apresentar informações de forma adequada. Navegadores web, aplicativos móveis e aplicações desktop conectadas à Internet são exemplos de clientes. Em uma arquitetura web, o cliente normalmente executa a camada de apresentação e parte da lógica de interface.

O cliente é o componente responsável pela interação direta com o usuário no modelo cliente-servidor. Suas funções incluem capturar ações, enviar requisições ao servidor, receber respostas e apresentar informações por meio de interfaces gráficas. Navegadores web, aplicativos móveis, aplicações desktop e consumidores de APIs são exemplos de clientes. Além disso, o cliente pode executar tarefas como renderização de páginas, execução de JavaScript, armazenamento de cookies e validações iniciais de dados, embora as verificações de segurança devam ser realizadas no servidor.

Os clientes podem ser classificados em clientes leves (Thin Clients) e clientes pesados (Fat Clients). Os clientes leves dependem fortemente do servidor para processamento, atuando principalmente como interfaces de acesso aos recursos da aplicação. Já os clientes pesados executam uma parcela significativa da lógica e do processamento localmente, reduzindo a dependência contínua do servidor. Aplicações desktop, aplicativos móveis avançados e Single Page Applications (SPAs) são exemplos comuns dessa segunda abordagem.



Servidor

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

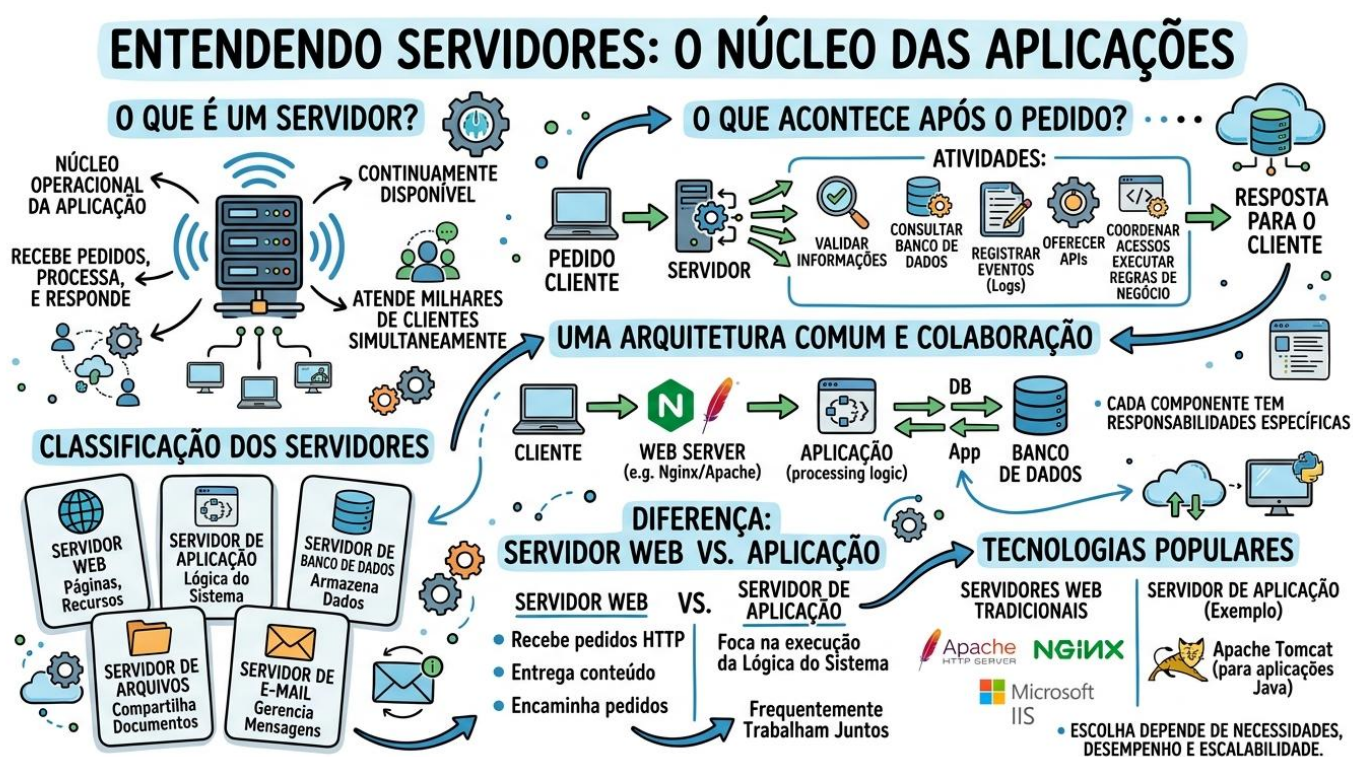
SERVIDOR

O servidor é o componente responsável por receber requisições dos clientes, processá-las e fornecer respostas adequadas. Ele concentra funcionalidades como regras de negócio, autenticação, acesso a bancos de dados, processamento de dados e integração com outros sistemas. Um único servidor pode atender simultaneamente milhares de clientes, funcionando como o núcleo operacional da aplicação. Em ambientes modernos, os

servidores frequentemente são distribuídos em múltiplas máquinas para aumentar disponibilidade e escalabilidade.

O servidor é o componente responsável por disponibilizar serviços aos clientes na arquitetura cliente-servidor. Ele permanece continuamente disponível para receber requisições, processar informações e retornar respostas adequadas. Entre suas funções estão validar dados, executar regras de negócio, consultar bancos de dados, registrar eventos em logs, disponibilizar APIs e coordenar o acesso a recursos compartilhados. Em arquiteturas mais completas, diferentes componentes colaboram para atender às solicitações, distribuindo responsabilidades entre servidor web, aplicação e banco de dados.

Os servidores podem ser classificados conforme sua função, incluindo servidores web, de aplicação, de banco de dados, de arquivos e de e-mail. Uma distinção importante é que o servidor web normalmente recebe requisições HTTP e entrega conteúdos ou encaminha solicitações, enquanto o servidor de aplicação é especializado na execução da lógica de negócio. Tecnologias como Apache HTTP Server, Nginx e Microsoft IIS são exemplos de servidores web, enquanto o Apache Tomcat é amplamente utilizado como servidor de aplicação para ambientes Java.



Requisição/Resposta

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

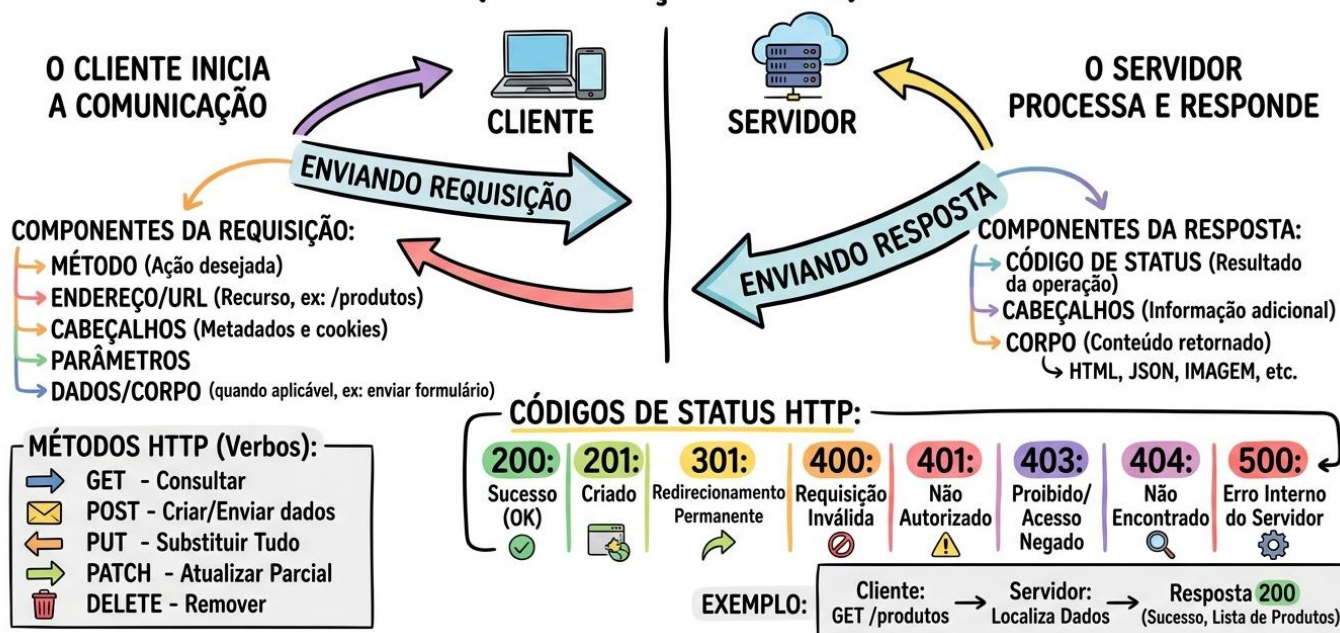
REQUISIÇÃO/RESPOSTA

O modelo requisição/resposta é o mecanismo básico de comunicação utilizado na Web. Nesse modelo, um cliente inicia a comunicação enviando uma requisição contendo informações como método, endereço, parâmetros e dados. O servidor processa a solicitação e retorna uma resposta contendo código de status, cabeçalhos e, normalmente, algum conteúdo. Esse padrão é amplamente utilizado em protocolos como HTTP e HTTPS e constitui a base da comunicação entre navegadores, APIs e serviços web.

A comunicação entre cliente e servidor na Web ocorre por meio do modelo de requisição e resposta. Quando o usuário realiza uma ação, o cliente envia uma requisição HTTP contendo informações como método, URL, cabeçalhos, cookies e, quando necessário, dados no corpo da mensagem. Os métodos HTTP indicam a operação desejada sobre um recurso, sendo os mais comuns GET para consultas, POST para criação ou envio de dados, PUT para substituição, PATCH para atualização parcial e DELETE para remoção.

Após receber a requisição, o servidor processa a solicitação e retorna uma resposta composta por código de status, cabeçalhos e conteúdo. Os códigos de status informam o resultado da operação, como 200 (sucesso), 201 (criado), 404 (não encontrado) e 500 (erro interno do servidor). Esse ciclo de requisição e resposta ocorre continuamente durante a navegação em aplicações web, permitindo a troca padronizada e organizada de informações entre clientes e servidores.

MODELO REQUISIÇÃO E RESPOSTA NA WEB (Comunicação na Web)



Sessão

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

SESSÃO

Uma sessão é um mecanismo utilizado para manter informações sobre um usuário ao longo de múltiplas requisições realizadas durante sua interação com uma aplicação. Como o protocolo HTTP é naturalmente sem estado (stateless), o servidor utiliza identificadores de sessão para associar diversas requisições ao mesmo usuário. Informações como autenticação, carrinho de compras, preferências temporárias e contexto de navegação costumam ser armazenadas na sessão até que ela expire ou seja encerrada.

A sessão é um mecanismo utilizado para manter o contexto de um usuário entre múltiplas requisições em aplicações web. Como o protocolo HTTP é stateless, o servidor não se lembra automaticamente de solicitações anteriores. Para contornar essa limitação, após a autenticação é criada uma sessão identificada por um ID único, geralmente enviado ao cliente por meio de um cookie. Nas requisições seguintes, esse identificador permite que o servidor recupere as informações associadas ao usuário e mantenha a continuidade da navegação.

As sessões podem armazenar dados como identidade do usuário autenticado, permissões de acesso, preferências temporárias e informações de carrinhos de compras. Sua principal vantagem é preservar o estado da aplicação sem exigir que o usuário se autentique a cada interação. Entretanto, como os dados costumam ficar armazenados no servidor, podem gerar consumo adicional de recursos e desafios de escalabilidade em ambientes distribuídos, o que frequentemente leva ao uso de armazenamentos compartilhados ou mecanismos baseados em tokens.



Cookies

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

COOKIES

Cookies são pequenos arquivos de texto armazenados pelo navegador a pedido de um servidor web. Eles permitem que informações sejam preservadas entre diferentes requisições, auxiliando na identificação de usuários, manutenção de sessões, armazenamento de preferências e personalização da experiência de navegação. Os cookies podem ser temporários ou persistentes e frequentemente trabalham em conjunto com mecanismos de autenticação e gerenciamento de sessões.

Cookies são pequenos dados armazenados no navegador do usuário para preservar informações entre diferentes interações com uma aplicação web. Eles podem armazenar preferências, identificadores de sessão e outras configurações necessárias ao funcionamento do sistema. Após serem enviados pelo servidor, os cookies passam a ser transmitidos automaticamente pelo navegador em requisições futuras para o mesmo domínio, permitindo que a aplicação reconheça informações previamente associadas ao usuário.

Os cookies podem ser temporários (de sessão) ou persistentes, além de possuir atributos de segurança como Secure, que restringe seu envio a conexões HTTPS, e HttpOnly, que impede o acesso por scripts JavaScript. Embora frequentemente utilizados em conjunto, cookies e sessões não são a mesma coisa: a sessão normalmente armazena os dados no servidor, enquanto o cookie permanece no navegador. Em

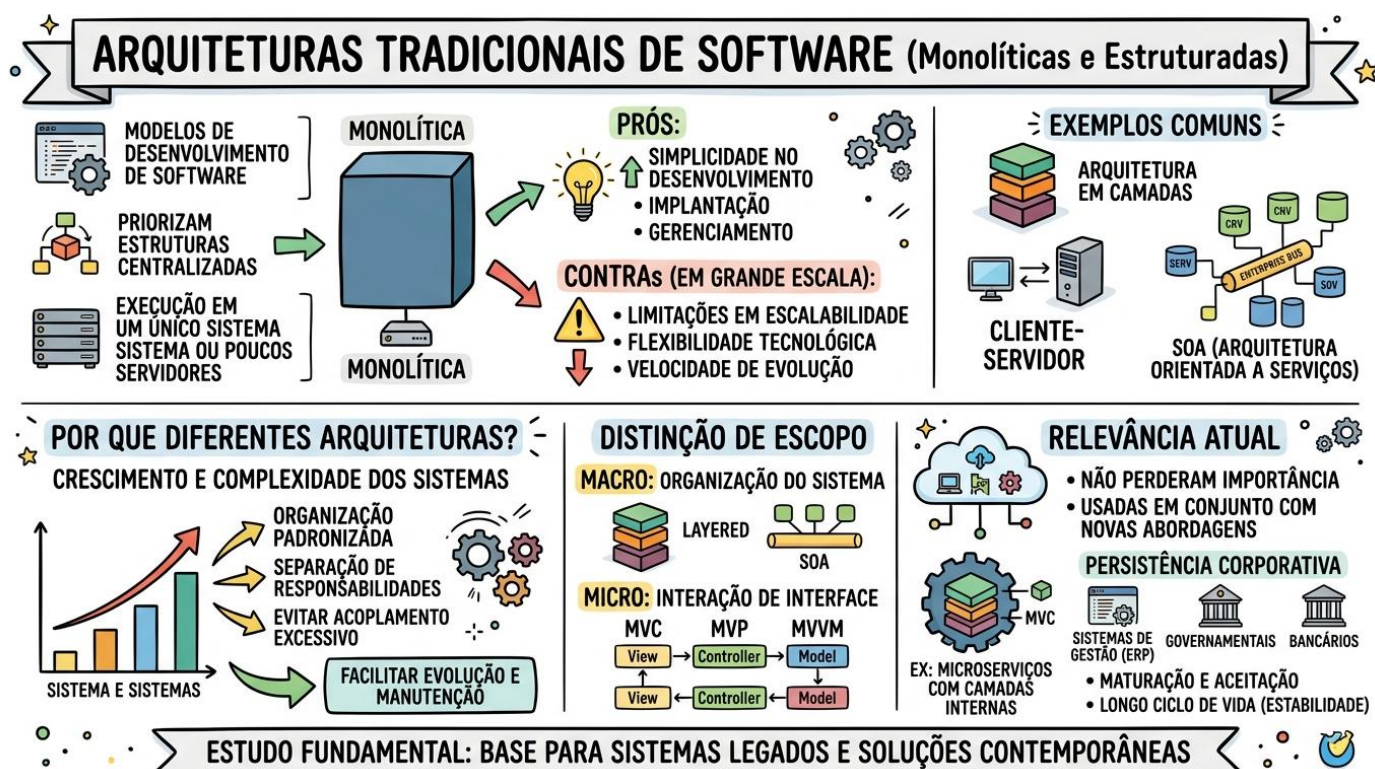
muitas aplicações, o cookie contém apenas o identificador que permite ao servidor localizar e recuperar a sessão correspondente ao usuário autenticado.



Arquiteturas Tradicionais

ARQUITETURAS TRADICIONAIS

Modelos de desenvolvimento de software que priorizam estruturas centralizadas e relativamente monolíticas, nas quais grande parte das funcionalidades da aplicação é executada em um único sistema ou conjunto reduzido de servidores. Exemplos comuns incluem aplicações monolíticas, arquiteturas cliente-servidor clássicas e arquiteturas em camadas. Essas abordagens costumam oferecer simplicidade de desenvolvimento, implantação e gerenciamento, porém podem apresentar limitações relacionadas à escalabilidade, flexibilidade tecnológica e velocidade de evolução quando utilizadas em sistemas de grande porte.



As arquiteturas tradicionais surgiram para organizar aplicações de forma estruturada, promovendo a separação de responsabilidades e reduzindo a complexidade dos sistemas. Modelos como Arquitetura em Camadas, MVC, MVP, MVVM e SOA foram desenvolvidos para facilitar manutenção, reutilização de componentes e evolução do software à medida que as aplicações se tornavam maiores e mais complexas.

Embora frequentemente estudados em conjunto, esses modelos possuem objetivos distintos. A Arquitetura em Camadas e a SOA definem formas amplas de organização dos sistemas, enquanto MVC, MVP e MVVM são padrões voltados principalmente para estruturar a interação entre interface, dados e lógica de apresentação. Essa diferença permite que múltiplos padrões coexistam dentro da mesma aplicação.

Mesmo com o surgimento de microserviços, computação em nuvem e arquiteturas orientadas a eventos, as arquiteturas tradicionais continuam amplamente utilizadas. Sistemas corporativos, governamentais e bancários frequentemente adotam esses modelos devido à sua maturidade e estabilidade. Além disso, muitos conceitos presentes nas arquiteturas modernas derivam desses padrões, tornando seu estudo fundamental para compreender tanto sistemas legados quanto soluções contemporâneas.

Arquitetura em Camadas

INCIDÊNCIA EM PROVA: ALTÍSSIMA

ARQUITETURA EM CAMADAS

A Arquitetura em Camadas organiza uma aplicação em níveis especializados, cada um responsável por um conjunto específico de funcionalidades. As camadas mais comuns são apresentação, negócio e dados. Essa separação permite que alterações em uma camada tenham impacto reduzido nas demais, aumentando a manutenibilidade, reutilização e organização do sistema. Trata-se de uma das arquiteturas mais utilizadas em sistemas corporativos devido à sua simplicidade e clareza estrutural.

A Arquitetura em Camadas, também conhecida como Multi-Tier Architecture ou N-Tier Architecture, é um dos modelos arquiteturais mais tradicionais e amplamente utilizados no desenvolvimento de software. Seu principal objetivo consiste em organizar a aplicação por meio da separação de responsabilidades, evitando que funcionalidades relacionadas à interface, às regras de negócio, ao processamento das operações e ao acesso aos dados fiquem concentradas em um único componente. Essa abordagem tornou-se especialmente popular em sistemas corporativos porque facilita a manutenção, melhora a organização do código e permite que a aplicação evolua de forma mais controlada ao longo do tempo.

ARQUITETURA EM CAMADAS



Por que arquitetos de software dedicam tanta atenção à separação de responsabilidades? Quando diferentes preocupações da aplicação são agrupadas em uma única estrutura, qualquer alteração tende a gerar impactos mais amplos e difíceis de prever. A Arquitetura em Camadas procura resolver esse problema distribuindo responsabilidades em níveis especializados. Dessa forma, cada parte do sistema possui um propósito claramente definido e pode evoluir com menor interferência sobre as demais.

Como essa divisão é realizada dentro da aplicação? Em vez de centralizar todo o processamento em uma única estrutura, o sistema é segmentado em camadas com funções específicas. De forma geral, encontram-se camadas responsáveis pela apresentação, pela coordenação dos processos da aplicação, pelas regras de negócio, pelo acesso aos dados e pelo armazenamento das informações. Cada uma delas contribui para o funcionamento do sistema sem assumir responsabilidades que pertencem às demais.

Em aplicações construídas segundo esse modelo, existe uma hierarquia lógica bem definida. As camadas mais externas normalmente estão associadas à interação com usuários e à apresentação das informações. À medida que se avança para o interior da arquitetura, surgem componentes responsáveis pela coordenação dos fluxos de processamento, pela aplicação das regras do domínio e pelo acesso aos mecanismos de persistência. Nas camadas mais internas encontram-se os recursos diretamente ligados à infraestrutura e ao armazenamento permanente das informações.

A Camada de Apresentação é responsável pela interação entre o sistema e seus usuários. Nela encontram-se páginas web, aplicações móveis, interfaces desktop e demais mecanismos utilizados para exibir informações e receber dados de entrada. Quando um usuário realiza uma ação, a solicitação é encaminhada para as camadas responsáveis pelo processamento da operação solicitada. Essa camada concentra-se na experiência de interação e na exibição dos resultados produzidos pelo sistema.

O que diferencia a Camada de Apresentação das demais camadas? Sua principal preocupação não está relacionada à execução de regras de negócio nem ao armazenamento de informações. Sua função consiste em receber interações, apresentar dados e encaminhar solicitações. Essa separação permite que alterações visuais sejam realizadas sem exigir mudanças nas regras que governam o funcionamento do domínio da aplicação.

A Camada de Aplicação tem como principal função coordenar o fluxo das operações. Ela recebe solicitações originadas na camada de apresentação, executa casos de uso, orquestra processos e determina quais componentes devem participar do atendimento da requisição. Sua responsabilidade não consiste em implementar diretamente as regras do negócio, mas coordenar sua execução de forma organizada e previsível.

Na Arquitetura em Camadas, a camada de aplicação é frequentemente projetada com foco em desacoplamento e independência de estado (stateless). Isso significa que ela procura evitar o armazenamento permanente do contexto das sessões dos usuários entre diferentes requisições. Quando informações de estado precisam ser mantidas, essa responsabilidade normalmente é transferida para bancos de dados, sistemas de cache distribuído ou mecanismos baseados em tokens.

Por que a característica stateless é tão valorizada? Porque ela favorece a escalabilidade horizontal da aplicação. Como as instâncias não dependem de informações armazenadas localmente em memória, qualquer servidor disponível pode processar uma requisição recebida. Essa característica simplifica a distribuição de carga e permite aumentar a capacidade do sistema por meio da adição de novas instâncias.

Uma dúvida bastante comum envolve a distinção entre Camada de Aplicação e Camada de Negócio. Não seriam ambas responsáveis pelo processamento do sistema? Embora exista colaboração entre elas, seus objetivos são diferentes. A Camada de Aplicação coordena o fluxo das operações e controla a execução dos casos de uso. Já a Camada de Negócio concentra as regras que representam o conhecimento do domínio e determinam como a organização funciona.

A Camada de Negócio representa o núcleo conceitual da aplicação. Nela encontram-se regras relacionadas a cálculos tributários, aprovação de pedidos, concessão de empréstimos, aplicação de descontos, validações de contratos e diversas outras operações que refletem diretamente as necessidades da organização. Alterar essa camada significa alterar o comportamento do domínio do sistema.

Por que essa separação entre aplicação e negócio é considerada tão importante? Porque a coordenação de processos e a definição das regras do domínio são preocupações distintas. Quando ambas são

misturadas, a manutenção torna-se mais difícil e as responsabilidades deixam de estar claramente delimitadas. A divisão entre essas camadas favorece a organização e reduz o acoplamento interno da aplicação.

A Camada de Persistência é responsável pela comunicação com os mecanismos de armazenamento. Componentes como DAOs, Repositories e ferramentas ORM realizam operações de consulta, inserção, atualização e remoção de dados. Essa camada abstrai os detalhes técnicos de acesso às informações, permitindo que as regras de negócio permaneçam independentes da tecnologia utilizada para persistência.

Já a Camada de Dados corresponde ao ambiente onde as informações são efetivamente armazenadas. Em muitos sistemas, essa camada é representada por bancos de dados relacionais como PostgreSQL, Oracle, SQL Server e MySQL. Entretanto, outras tecnologias de persistência também podem ser utilizadas, incluindo bancos NoSQL, mecanismos de armazenamento distribuído e soluções especializadas para diferentes cenários.

O que acontece quando uma requisição percorre toda a arquitetura? Uma ação iniciada pelo usuário na interface atravessa a camada de aplicação, aciona as regras de negócio, realiza consultas ou alterações por meio da camada de persistência e acessa os mecanismos de armazenamento quando necessário. Após o processamento, os resultados retornam gradualmente até a camada de apresentação, onde são exibidos ao usuário. Essa organização favorece a previsibilidade e torna mais simples compreender o fluxo interno da aplicação.

Uma característica importante das arquiteturas em camadas é a existência de mecanismos de abstração que isolam diferentes níveis do sistema. As camadas superiores trabalham com conceitos relacionados ao domínio da aplicação e à interação com os usuários, enquanto as camadas inferiores concentram detalhes técnicos necessários para a execução das operações. Essa divisão reduz a dependência entre componentes e contribui para uma arquitetura mais organizada e flexível.

Por que essa separação é considerada tão importante? Um dos principais objetivos da Arquitetura em Camadas consiste em impedir que regras de negócio dependam diretamente de aspectos relacionados à infraestrutura. Dessa forma, componentes responsáveis pelo domínio da aplicação podem concentrar-se na resolução dos problemas do negócio sem precisar conhecer detalhes sobre sistemas operacionais, bancos de dados, protocolos de rede ou mecanismos de armazenamento utilizados pela plataforma.

À medida que avançamos das camadas externas para as camadas internas, observamos uma mudança gradual de responsabilidade. As camadas mais próximas do usuário tratam da apresentação das informações e da recepção das solicitações. Em níveis mais profundos encontram-se componentes voltados à coordenação dos processos, à execução das regras de negócio e ao acesso aos recursos de infraestrutura. Essa organização cria uma cadeia de abstrações que reduz o acoplamento entre diferentes partes do sistema.

Uma dúvida comum surge nesse ponto: as camadas mais internas comunicam-se diretamente com o sistema operacional? Em muitos casos, elas são as camadas mais próximas dos recursos de infraestrutura e dos serviços disponibilizados pela plataforma de execução. Entretanto, essa comunicação frequentemente ocorre por intermédio de bibliotecas, frameworks, runtimes e APIs especializadas. Em aplicações modernas, raramente as regras da aplicação acessam diretamente os mecanismos internos do sistema operacional.

Essas camadas mais profundas normalmente concentram responsabilidades relacionadas ao acesso a arquivos, comunicação de rede, utilização de bancos de dados, integração com serviços externos e consumo de recursos disponibilizados pelo ambiente de execução. Como consequência, as camadas

superiores permanecem protegidas contra mudanças em tecnologias específicas de infraestrutura. Uma alteração no mecanismo de persistência, por exemplo, tende a produzir impactos menores quando existe uma camada apropriada de abstração.

Pode-se afirmar, portanto, que as camadas funcionam como barreiras de proteção entre o domínio da aplicação e os detalhes operacionais do ambiente onde ela é executada. Essa estrutura permite que regras de negócio permaneçam relativamente independentes de hardware, sistemas operacionais e tecnologias específicas. Ao concentrar as dependências técnicas em níveis mais internos da arquitetura, torna-se mais simples manter, evoluir e adaptar o sistema a novos requisitos sem comprometer os componentes centrais responsáveis pelo comportamento do negócio.

Uma das implementações mais conhecidas desse modelo é a arquitetura de três camadas, conhecida como Three-Tier Architecture. Ela normalmente é composta por apresentação, aplicação e dados. Isso significa que toda arquitetura multicamadas possui exatamente três níveis? Não. O conceito de Multi-Tier é muito mais amplo e permite a existência de quantas camadas forem necessárias para atender às necessidades arquiteturais do sistema.

A arquitetura de três camadas tornou-se extremamente popular em aplicações corporativas porque oferece um equilíbrio entre simplicidade e organização. Durante muitos anos, esse modelo dominou o desenvolvimento de aplicações empresariais, especialmente em ambientes web. Ainda hoje ele continua presente em inúmeros sistemas legados e em diversas soluções modernas.

Além do modelo tradicional de três camadas, existe também a arquitetura de quatro camadas, conhecida como 4-Tier Architecture. Nessa configuração, parte das responsabilidades intermediárias é separada em níveis distintos para aumentar a segurança, melhorar a escalabilidade e distribuir melhor o processamento. Como resultado, surge uma estrutura composta por Camada de Cliente ou Apresentação, Camada Web, Camada de Aplicação ou Negócio e Camada de Dados.

Qual é o papel específico da Camada Web nesse modelo? Ela costuma ser responsável pelo tratamento das requisições HTTP, pela entrega de conteúdo estático e pelo gerenciamento das conexões entre clientes e servidores. Enquanto isso, a Camada de Aplicação permanece dedicada à execução dos processos e das regras de negócio. Essa separação reduz a exposição dos componentes internos e contribui para uma distribuição mais adequada das responsabilidades.

Entre os benefícios da Arquitetura em Camadas destacam-se a separação de responsabilidades, a reutilização das regras de negócio, a melhoria da testabilidade, a facilidade de manutenção e o aumento da segurança. Como o acesso aos dados ocorre de forma controlada, torna-se mais simples implementar mecanismos de proteção e governança das informações. Além disso, a divisão em níveis especializados favorece a compreensão da estrutura do sistema.

Outro benefício importante está relacionado à reutilização. Como as regras de negócio permanecem separadas das interfaces, diferentes aplicações podem compartilhar os mesmos componentes. Uma aplicação web, um aplicativo móvel e uma API podem utilizar a mesma lógica de domínio sem necessidade de duplicação de código.

Entretanto, esse modelo não está livre de limitações. Dependendo da forma como é implementado, uma solicitação simples pode atravessar diversas camadas antes de atingir o banco de dados. Esse percurso aumenta a quantidade de chamadas internas e pode introduzir algum custo adicional de processamento. Ainda assim, para muitos sistemas corporativos, os benefícios organizacionais superam amplamente esse custo.

Embora seja tecnicamente mais comum considerar o MVC como um padrão localizado dentro da camada de apresentação, é frequente encontrar em materiais acadêmicos, arquiteturas tradicionais e provas de concursos públicos uma associação entre MVC e Arquitetura de Três Camadas. Nessa interpretação, os componentes do MVC são mapeados para diferentes níveis da estrutura arquitetural.

Como ocorre esse mapeamento? A camada de apresentação é associada à View. A camada responsável pelo controle do fluxo da aplicação é relacionada ao Controller. Já a camada vinculada ao domínio e ao acesso aos dados é associada ao Model. Essa equivalência possui valor didático e continua aparecendo em diversos contextos educacionais, embora não represente necessariamente a interpretação arquitetural mais moderna do padrão MVC.

Além da definição das responsabilidades de cada camada, existe uma decisão arquitetural importante relacionada ao modo como elas podem se comunicar. Essa decisão envolve os conceitos de camada fechada e camada aberta. A escolha influencia diretamente aspectos como isolamento arquitetural, desempenho, acoplamento e facilidade de manutenção.

O que caracteriza uma camada fechada? Nesse modelo, uma camada só pode interagir diretamente com a camada imediatamente inferior. Isso significa que uma solicitação deve percorrer todas as etapas previstas pela arquitetura, respeitando rigorosamente a sequência estabelecida. Essa restrição aumenta o isolamento entre os componentes e fortalece a separação de responsabilidades.

As camadas fechadas oferecem um maior controle arquitetural porque dificultam atalhos que poderiam comprometer a organização do sistema. Como consequência disso, tornam-se especialmente adequadas para aplicações que priorizam manutenibilidade, rastreabilidade e consistência estrutural ao longo do tempo.

Entretanto, existem situações em que essa regra pode ser flexibilizada. Surge então o conceito de camada aberta. Nesse modelo, determinadas requisições podem acessar diretamente camadas mais inferiores sem passar por todas as intermediárias. Essa estratégia pode reduzir etapas de processamento e melhorar o desempenho de determinados fluxos da aplicação.

Essa flexibilidade possui algum custo? Sim. Embora possa trazer ganhos de desempenho em cenários específicos, também aumenta as dependências entre componentes e reduz parte do isolamento proporcionado pela arquitetura em camadas. Como resultado, a manutenção pode tornar-se mais complexa caso o uso dessas exceções não seja cuidadosamente controlado.

Qual abordagem deve ser utilizada? Não existe uma resposta universal. Camadas fechadas favorecem organização, controle e previsibilidade. Camadas abertas podem ser justificadas quando existem requisitos de desempenho que tornam inadequada a passagem obrigatória por todos os níveis intermediários. A escolha depende dos objetivos arquiteturais e dos compromissos considerados aceitáveis para o projeto.

Outro conceito importante associado a esse modelo é o antipadrão Sinkhole. Esse problema ocorre quando uma solicitação atravessa várias camadas consecutivas sem que elas executem qualquer processamento significativo. Em vez de aplicar validações, regras de negócio, transformações ou coordenação relevante, essas camadas apenas encaminham a requisição para o próximo componente da cadeia.

Por que esse cenário é considerado um antipadrão? Porque a arquitetura passa a introduzir custo operacional sem fornecer benefícios equivalentes. O sistema continua percorrendo múltiplos níveis, mas esses níveis deixam de agregar responsabilidades reais ao processamento. Como consequência, a complexidade aumenta sem produzir ganhos efetivos de organização.

Isso significa que toda delegação entre camadas caracteriza um Sinkhole? Não. A comunicação entre camadas é uma característica natural desse estilo arquitetural. O problema surge quando uma parcela significativa das requisições atravessa sucessivas camadas que não executam nenhuma atividade relevante. Nesses casos, a estrutura arquitetural deixa de contribuir para a organização do sistema e passa a representar apenas sobrecarga e complexidade adicional.

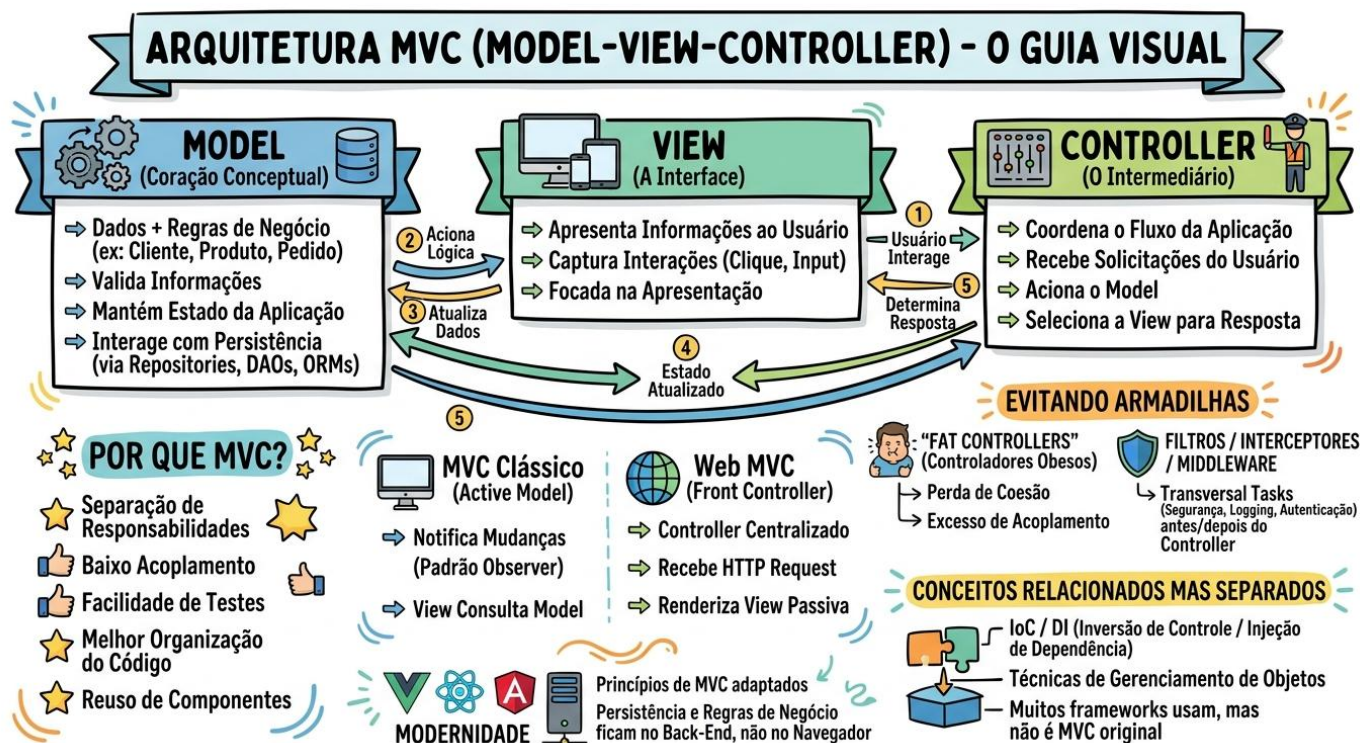
A compreensão desses conceitos permite analisar a Arquitetura em Camadas de forma mais completa. Não se trata apenas de dividir a aplicação em níveis distintos, mas de definir responsabilidades adequadas, estabelecer mecanismos de comunicação coerentes e equilibrar fatores como desempenho, manutenção, escalabilidade e organização. É justamente essa combinação de princípios que explica por que esse modelo continua sendo uma das arquiteturas mais importantes da Engenharia de Software.

Arquitetura MVC

INCIDÊNCIA EM PROVA: ALTÍSSIMA

ARQUITETURA MVC

MVC (Model-View-Controller) é um padrão arquitetural que divide a aplicação em três componentes principais. O Model representa os dados e as regras de negócio, a View é responsável pela interface apresentada ao usuário e o Controller atua como intermediário, recebendo entradas do usuário e coordenando as ações necessárias. Essa separação de responsabilidades melhora a organização do código, facilita testes e reduz o acoplamento entre interface e lógica de negócio.



MVC (Model-View-Controller) é um padrão arquitetural criado para organizar aplicações por meio da separação de responsabilidades. Seu objetivo é dividir o sistema em componentes especializados, reduzindo o acoplamento entre interface, lógica de negócio e controle da aplicação. Proposto por Trygve Reenskaug na década de 1970, esse modelo tornou-se uma das abordagens mais influentes no desenvolvimento de software e continua presente em diversos frameworks modernos.

Como o MVC consegue organizar uma aplicação de forma mais estruturada? A resposta está na divisão do sistema em três componentes principais. O Model representa os dados e as regras de negócio; a View é responsável pela apresentação das informações ao usuário; e o Controller coordena o fluxo da aplicação. Cada componente possui atribuições específicas, o que facilita a manutenção e reduz a dependência entre diferentes partes do sistema.

O Model concentra os elementos relacionados ao domínio da aplicação. Nele são definidas estruturas como Cliente, Conta, Pedido e Produto, além das regras que determinam o comportamento desses elementos. Sua responsabilidade inclui validar informações, manipular dados e representar os conceitos centrais do negócio. Em muitas implementações, o Model também interage com mecanismos responsáveis pela persistência dos dados armazenados.

Na literatura de Engenharia de Software aplicada ao desenvolvimento de aplicações, o Model é frequentemente considerado o núcleo conceitual da arquitetura MVC. Sua função vai muito além do simples armazenamento de dados. Ele concentra informações, estados relevantes do sistema e regras que determinam o comportamento dos elementos centrais do negócio.

O que significa afirmar que o Model representa o domínio da aplicação? Significa que ele reúne estruturas responsáveis por descrever conceitos fundamentais do sistema, como clientes, contas, pedidos, produtos e demais entidades de negócio. Essas estruturas não existem apenas para armazenar informações. Elas também incorporam comportamentos, restrições e regras que refletem o funcionamento esperado do domínio representado pela aplicação.

Além de representar os dados, o Model também participa da gestão do estado da aplicação. Como o sistema sabe que um pedido foi aprovado, que uma conta foi bloqueada ou que um produto está indisponível? Essas informações fazem parte do estado lógico dos objetos do domínio. O Model é responsável por manter e atualizar esse estado de acordo com as regras estabelecidas para o funcionamento do negócio.

No conceito mais puro do padrão MVC, especialmente em aplicações desktop orientadas a objetos, o Model não atua apenas de forma passiva armazenando entidades de negócio. Ele é o componente responsável por manter o estado atual da aplicação. Quando operações executadas pelo sistema modificam dados ou regras lógicas, o estado interno do Model é atualizado. Cabe então ao próprio Model disponibilizar essas informações atualizadas para que a interface reflita corretamente a situação corrente da aplicação.

Como ocorre a interação com fontes externas de dados? Em muitas implementações de MVC, o Model está associado aos componentes responsáveis pela persistência das informações. Em arquiteturas mais modernas, entretanto, essa comunicação frequentemente é realizada por mecanismos especializados, como Repositories, DAOs e ferramentas ORM. Independentemente da estratégia adotada, o objetivo permanece o mesmo: manter View e Controller desacoplados das responsabilidades relacionadas ao armazenamento e à recuperação dos dados.

A View desempenha o papel de interface entre o sistema e o usuário. Sua função é apresentar informações e capturar interações realizadas pelo usuário. Páginas HTML, telas de aplicativos móveis e interfaces desktop são exemplos típicos desse componente. A View deve concentrar-se na apresentação dos dados, evitando incorporar regras de negócio que pertencem a outras partes da arquitetura.

Uma característica importante do MVC consiste justamente na separação das responsabilidades relacionadas aos dados e à apresentação. Por que a View não deve manipular diretamente informações do domínio? Porque sua função é apresentar informações ao usuário e capturar interações. Quando

regras de negócio e controle de estado são transferidos para a interface, ocorre uma mistura de responsabilidades que aumenta o acoplamento e dificulta a manutenção da aplicação.

Em aplicações desktop tradicionais, a camada de View é frequentemente composta por classes de interface gráfica associadas a contêineres visuais. Esses contêineres não apenas organizam componentes de tela, como também detectam eventos provenientes de dispositivos de entrada, como mouse e teclado. Embora sejam responsáveis por capturar fisicamente as interações do usuário, sua função não é processar regras de negócio, mas encaminhar os eventos para o Controller apropriado.

O Controller atua como intermediário entre a interface e o domínio da aplicação. Quando uma ação é realizada pelo usuário, o Controller recebe a solicitação, coordena o processamento necessário e interage com o Model para executar as operações apropriadas. Após obter os resultados, ele determina qual View será utilizada para apresentar a resposta. Esse papel de coordenação permite manter separadas as responsabilidades de apresentação e negócio.

Da mesma forma que a View não deve assumir responsabilidades relacionadas ao domínio, o Controller também não deve atuar como repositório das informações da aplicação. Sua responsabilidade principal consiste em coordenar o fluxo das operações. Ao receber uma solicitação, ele aciona os componentes apropriados, coordena o processamento necessário e encaminha os resultados para apresentação. O Controller participa da execução do fluxo, mas não substitui o papel desempenhado pelo Model na representação do domínio.

A literatura clássica de Engenharia de Software descreve o MVC como uma arquitetura baseada na cooperação entre três componentes fundamentais: Model, View e Controller. Cada um desses elementos possui responsabilidades específicas e bem definidas, permitindo que a aplicação mantenha separadas as preocupações relacionadas aos dados, à apresentação e ao controle das interações. Essa divisão constitui um dos principais motivos para a ampla adoção do padrão ao longo das últimas décadas.

Como essa cooperação ocorre na prática? O fluxo de funcionamento do MVC normalmente inicia com uma interação do usuário na View. A solicitação é encaminhada ao Controller, que interpreta a ação realizada e determina quais operações devem ser executadas. Para isso, ele aciona os comportamentos apropriados do Model e coordena o processamento necessário. Em seguida, os resultados retornam ao Controller, que seleciona a View adequada para exibição das informações ao usuário.

Essa organização proporciona benefícios importantes, como facilidade de manutenção, reutilização de componentes, redução do acoplamento e melhor estruturação do sistema. Como cada elemento possui responsabilidades claramente delimitadas, torna-se possível evoluir partes da aplicação sem necessariamente produzir impactos significativos nas demais.

Entretanto, o funcionamento do MVC varia conforme o contexto em que ele é utilizado. Embora os princípios fundamentais permaneçam os mesmos, a forma como os componentes interagem pode sofrer adaptações de acordo com a plataforma, a linguagem ou o framework empregado.

Na formulação clássica do padrão, frequentemente chamada de MVC Clássico ou Active Model, existe uma relação mais dinâmica entre o Model e a View. Nesse modelo, o Model não apenas mantém os dados e o estado da aplicação, mas também participa ativamente da comunicação com as interfaces que dependem dessas informações.

Por que a View precisa acompanhar as mudanças ocorridas no Model? Sempre que o estado do domínio é alterado, a representação exibida ao usuário deve refletir essa mudança. Se um pedido for aprovado, uma conta for bloqueada ou um produto tornar-se indisponível, a interface precisa apresentar imediatamente essa nova situação ao usuário.

Para viabilizar esse comportamento, implementações clássicas do MVC frequentemente utilizam o padrão de projeto Observer. Quando ocorre uma alteração relevante no estado do Model, ele notifica os componentes interessados sobre essa mudança. Ao receber essa notificação, as Views atualizam sua representação visual consultando novamente as informações necessárias.

Nesse cenário, o Controller não é responsável por propagar alterações para as interfaces. Sua função continua sendo interpretar solicitações, coordenar o fluxo das operações e solicitar modificações apropriadas ao Model. Após a alteração do estado interno do domínio, a responsabilidade pela comunicação dessas mudanças passa a pertencer ao próprio Model.

Como as Views reagem a essas notificações? Em vez de receber diretamente todos os dados atualizados, elas normalmente utilizam a notificação apenas como um sinal de que o estado do Model mudou. A partir desse evento, cada View consulta novamente o Model para recuperar as informações necessárias e atualizar sua representação visual. Esse mecanismo permite que múltiplas interfaces permaneçam sincronizadas com o mesmo estado de domínio sem que o Model precise conhecer detalhes específicos de apresentação.

Um equívoco relativamente comum consiste em atribuir o baixo acoplamento do MVC exclusivamente ao mecanismo de notificação entre Model e View. Entretanto, a principal origem desse benefício está na separação clara das responsabilidades atribuídas a cada componente da arquitetura. O desacoplamento existe porque dados, apresentação e controle permanecem isolados em estruturas distintas.

Por que essa distinção é tão importante? Quando apresentação, regras de negócio e controle das interações permanecem separados, modificações realizadas em uma dessas áreas tendem a produzir menos impactos sobre as demais. A interface pode evoluir sem exigir alterações significativas nas regras do domínio, enquanto mudanças no comportamento do negócio podem ocorrer sem obrigar a reconstrução da camada de apresentação.

Embora essa descrição corresponda ao MVC clássico encontrado na literatura original, a realidade das aplicações web modernas costuma apresentar algumas diferenças importantes. No ecossistema Web, a arquitetura MVC normalmente assume uma forma mais centralizada em torno do Controller.

Em aplicações baseadas no padrão Front Controller, a navegação é predominantemente conduzida pelo controlador. Ele recebe a requisição HTTP, interpreta a solicitação, consulta o Model, executa os processos necessários e seleciona qual View será renderizada. Nesse contexto, a View tende a desempenhar um papel mais passivo, limitando-se à apresentação das informações fornecidas pelo controlador.

Essa diferença explica por que muitos desenvolvedores têm uma percepção distinta do MVC quando trabalham exclusivamente com aplicações web. Enquanto no MVC clássico a View frequentemente consulta diretamente o Model e reage às notificações de mudança de estado, em muitos frameworks web modernos o Controller atua como principal ponto de coordenação da aplicação.

A rígida separação estrutural proposta pelo MVC também favorece sua integração com outros padrões de projeto. Um exemplo interessante é o padrão comportamental Strategy. Em determinadas arquiteturas, diferentes Controllers podem ser associados a uma mesma View ou substituídos dinamicamente durante a execução.

Essa flexibilidade permite que a aplicação responda às mesmas interações de maneiras distintas sem exigir modificações na camada de apresentação. Dependendo do contexto, usuários diferentes, perfis

distintos ou regras específicas podem fazer com que controladores diferentes sejam utilizados para processar uma mesma interação.

Outro aspecto frequentemente associado ao MVC é a utilização de mecanismos de Inversão de Controle (IoC) e Injeção de Dependências (Dependency Injection - DI). Muitos desenvolvedores acabam concluindo que esses conceitos fazem parte do próprio padrão MVC. Entretanto, essa associação não é correta.

O MVC foi concebido para definir responsabilidades entre Model, View e Controller. Já IoC e DI são técnicas de construção de software destinadas a controlar a criação e o fornecimento de dependências entre objetos. Embora sejam amplamente utilizados em conjunto, tratam-se de conceitos independentes. Essa confusão ocorre porque diversos frameworks modernos implementam MVC juntamente com contêineres de IoC. Como resultado, Controllers, Services e outros componentes passam a ser criados, configurados e conectados automaticamente pela infraestrutura da plataforma.

Essa integração reduz a quantidade de código necessária para configurar a aplicação e simplifica a manutenção do sistema. Ainda assim, é importante compreender que tais recursos pertencem ao framework utilizado e não à definição original do padrão MVC. Em implementações mais tradicionais, a criação e associação dos componentes podem ser realizadas manualmente pelo próprio desenvolvedor. Em plataformas modernas, entretanto, essas tarefas costumam ser delegadas a contêineres especializados responsáveis por gerenciar o ciclo de vida dos objetos da aplicação.

À medida que um sistema cresce, surge outro desafio recorrente: os chamados fat controllers. Nessa situação, as classes controladoras passam a acumular responsabilidades excessivas, tornando-se extensas, difíceis de compreender e complicadas de manter. Em vez de apenas coordenar o fluxo da aplicação, esses controladores começam a concentrar validações, verificações de segurança, autenticação, autorização, geração de logs e diversas outras funções que não pertencem diretamente ao seu propósito principal. Como consequência, o controlador perde coesão e torna-se um ponto de acoplamento excessivo dentro do sistema.

Para evitar esse problema, arquiteturas modernas costumam introduzir mecanismos de interceptação, como filtros, interceptadores e middlewares. Esses componentes executam tarefas transversais antes ou depois do processamento realizado pelo Controller. Dessa forma, autenticação, autorização, controle de acesso, registro de logs, tratamento de exceções e outras preocupações comuns podem ser implementadas de forma centralizada. Com isso, os Controllers permanecem focados em sua responsabilidade original: coordenar a interação entre View e Model e conduzir o fluxo da aplicação.

Com a ascensão do desenvolvimento moderno para navegadores, bibliotecas e frameworks JavaScript declarativos, como Vue.js, React, Angular e Alpine.js, surgiram novas abordagens para organização das interfaces. Muitas dessas soluções utilizam adaptações do MVC ou padrões relacionados, como MVVM. Apesar das diferenças de implementação, os princípios fundamentais permanecem os mesmos. Continua existindo uma preocupação constante com a separação de responsabilidades, com o isolamento entre apresentação e regras de negócio e com a redução do acoplamento entre os componentes da aplicação.

Na prática, nenhuma dessas ferramentas transfere ao Controller executado no navegador a responsabilidade de acessar diretamente bancos de dados relacionais ou mecanismos definitivos de persistência. Essas operações continuam pertencendo aos componentes responsáveis pelo domínio e pela lógica de negócio hospedados no ambiente de back-end.

Essa permanência dos princípios fundamentais ajuda a explicar por que o MVC continua sendo um dos padrões arquiteturais mais importantes da Engenharia de Software. Mesmo após décadas de evolução

tecnológica, seus conceitos continuam influenciando a construção de aplicações web, desktop e móveis, servindo como base para diversos frameworks e arquiteturas utilizadas atualmente.

Arquitetura MVP

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

ARQUITETURA MVP

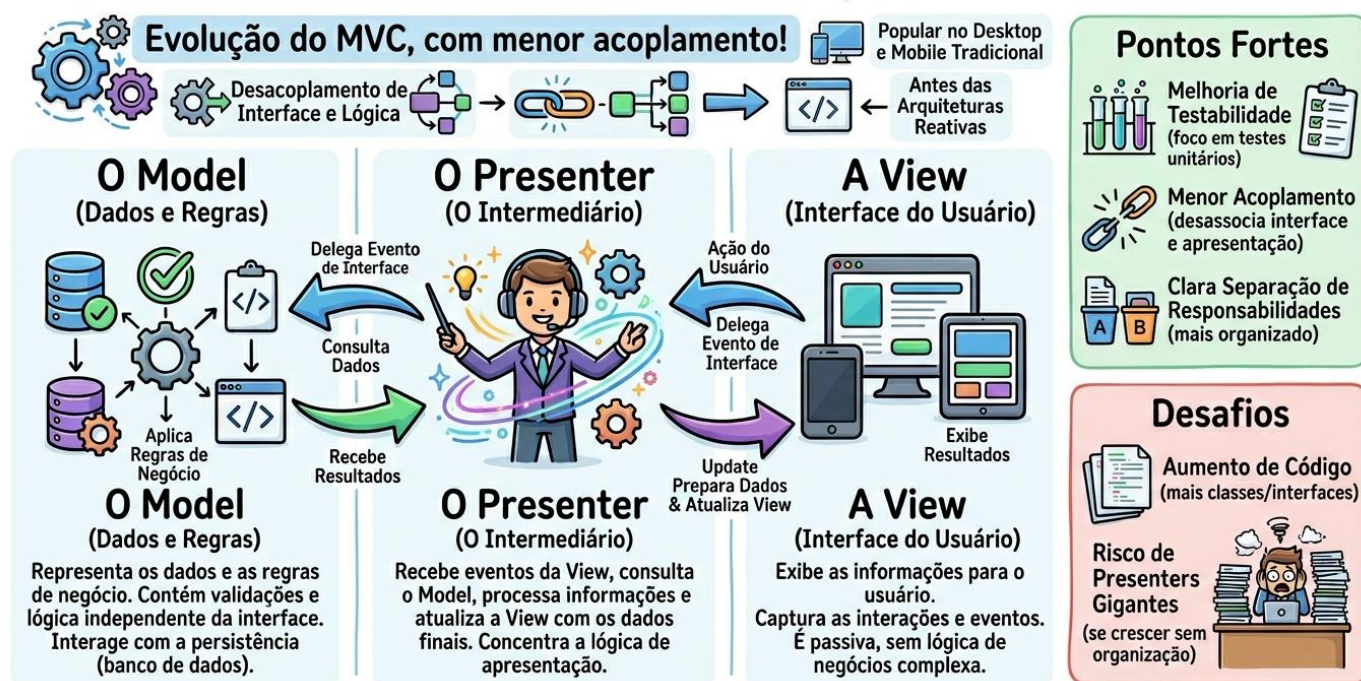
MVP (Model-View-Presenter) é uma evolução do MVC que busca aumentar o desacoplamento entre interface e lógica de apresentação. Nesse padrão, a View possui comportamento mais passivo e delega praticamente toda a lógica de interação ao Presenter. O Presenter recebe eventos da interface, processa as informações e atualiza a View. Essa abordagem facilita testes unitários e foi amplamente utilizada em aplicações desktop e mobile antes da popularização de arquiteturas reativas.

O MVP (Model-View-Presenter) é um padrão arquitetural criado para reduzir o acoplamento entre a interface do usuário e a lógica de apresentação. Seu objetivo é distribuir responsabilidades de forma organizada, facilitando a manutenção, a evolução e os testes da aplicação. Para isso, divide o sistema em três componentes principais: Model, View e Presenter.

O Model concentra os dados e as regras de negócio da aplicação, permanecendo independente da interface. A View é responsável por exibir informações e capturar ações do usuário, evitando incorporar lógica de negócio. Já o Presenter atua como intermediário entre esses componentes, recebendo eventos da View, consultando ou atualizando o Model e preparando os dados que serão exibidos na interface.

Entre as principais vantagens do MVP estão a melhoria da testabilidade, a redução do acoplamento e a separação mais clara das responsabilidades. Em contrapartida, o padrão pode aumentar a quantidade de código e gerar Presenters excessivamente complexos quando não há uma boa divisão de responsabilidades. Por essas características, o MVP foi amplamente utilizado em aplicações desktop, interfaces complexas e versões tradicionais do desenvolvimento Android.

MVP: Model-View-Presenter - Arquitetura de Software



Arquitetura MVVM

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

ARQUITETURA MVVM

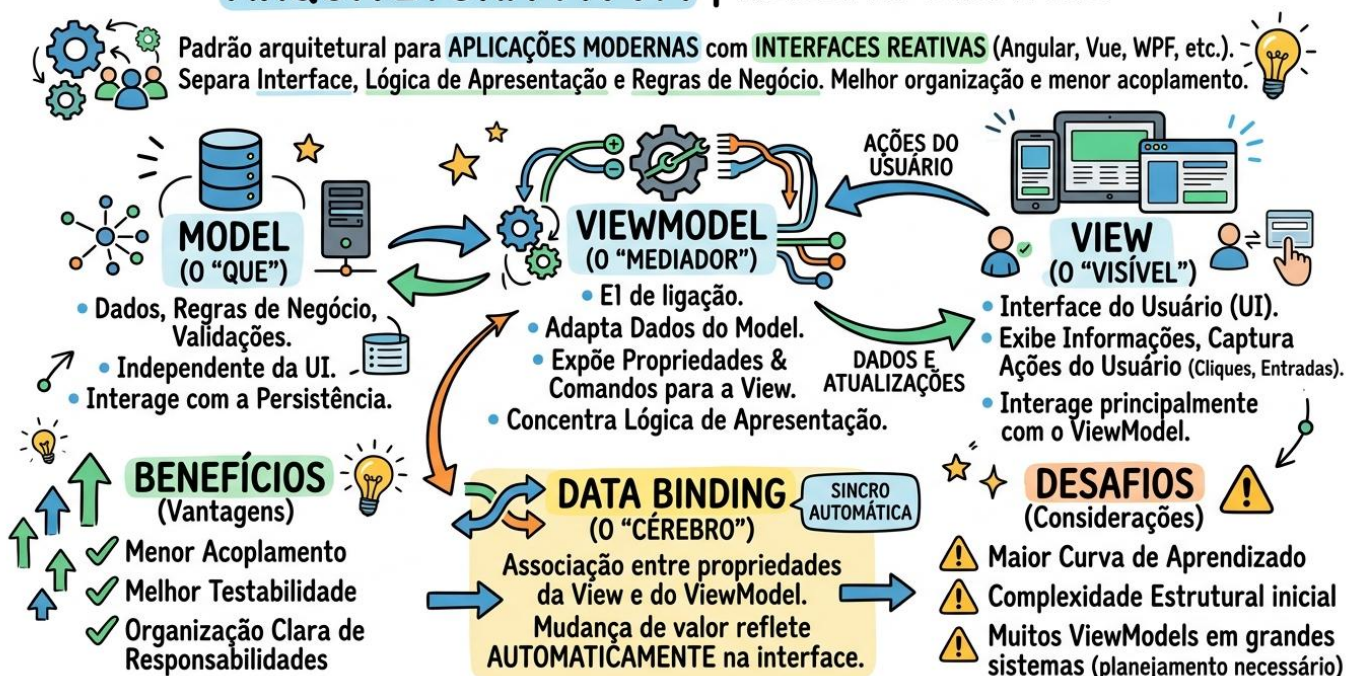
MVVM (Model-View-ViewModel) é um padrão arquitetural muito utilizado em aplicações modernas com interfaces reativas. O ViewModel atua como intermediário entre a View e o Model, expondo dados e comandos de forma que a interface possa se atualizar automaticamente por meio de mecanismos de data binding. Isso reduz a necessidade de manipulação direta da interface e melhora a separação de responsabilidades, sendo comum em frameworks como Angular, Vue, WPF e outras tecnologias orientadas a componentes.

O MVVM (Model-View-ViewModel) é um padrão arquitetural que busca separar de forma clara a interface gráfica, a lógica de apresentação e as regras de negócio. Popularizado em tecnologias como WPF e Silverlight, ele organiza a aplicação em três componentes principais: Model, responsável pelos dados e regras de negócio; View, responsável pela interface; e ViewModel, que atua como intermediário entre ambos.

O Model concentra as informações e comportamentos do domínio da aplicação, enquanto a View exibe os dados e captura as ações do usuário. O ViewModel adapta os dados provenientes do Model, disponibiliza propriedades para a interface e processa comandos originados na View. Dessa forma, a interface passa a interagir principalmente com o ViewModel, reduzindo o acoplamento direto com a lógica de negócio.

Uma das características mais marcantes do MVVM é o Data Binding, mecanismo que sincroniza automaticamente dados entre View e ViewModel. Isso reduz a necessidade de código para atualização da interface e facilita a manutenção da aplicação. Entre suas vantagens estão a melhor organização das responsabilidades, a redução do acoplamento e a maior testabilidade; como desvantagens, destacam-se a curva de aprendizado inicial e o aumento da complexidade estrutural em sistemas maiores.

ARQUITETURA MVVM | RESUMO DIDÁTICO



Arquitetura SOA

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

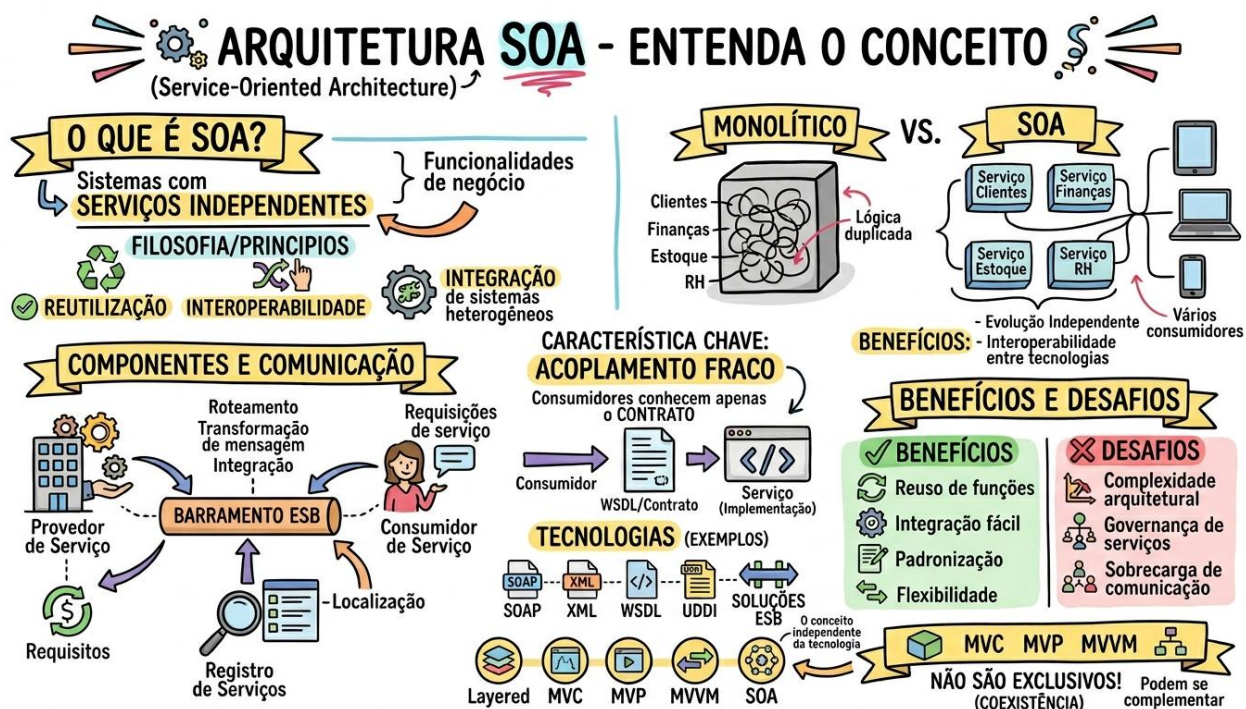
ARQUITETURA SOA

SOA (Service-Oriented Architecture) é um estilo arquitetural baseado na construção de sistemas compostos por serviços independentes que disponibilizam funcionalidades de negócio através de interfaces bem definidas. Esses serviços podem ser reutilizados por diferentes aplicações e frequentemente utilizam protocolos padronizados para comunicação. O principal objetivo da SOA é promover interoperabilidade, reutilização e integração entre sistemas corporativos heterogêneos.

SOA (Service-Oriented Architecture) é um estilo arquitetural que organiza sistemas em serviços independentes, reutilizáveis e acessíveis por diferentes aplicações. Seu principal objetivo é facilitar a integração entre sistemas corporativos, permitindo que funcionalidades como cadastro de clientes, estoque ou finanças sejam compartilhadas sem a necessidade de duplicação de código ou regras de negócio.

Na arquitetura SOA, os serviços são disponibilizados por provedores (Service Providers) e consumidos por aplicações ou componentes interessados (Service Consumers). Em muitas implementações, mecanismos de registro e barramentos de serviços (ESB) auxiliam na localização, integração, roteamento e transformação de mensagens entre os diversos sistemas participantes da arquitetura.

Uma das características mais importantes da SOA é o baixo acoplamento. Os consumidores precisam conhecer apenas os contratos de comunicação dos serviços, sem depender de detalhes internos de implementação. Isso favorece a interoperabilidade entre tecnologias diferentes e permite que os serviços evoluam de forma mais independente, mantendo a compatibilidade com seus consumidores.



Entre os benefícios da SOA estão a reutilização de funcionalidades, a integração de sistemas heterogêneos e a padronização da comunicação. Entretanto, a arquitetura também pode introduzir desafios relacionados à governança, à complexidade operacional e à sobrecarga de comunicação. Além disso, a SOA não substitui padrões como Arquitetura em Camadas, MVC, MVP ou MVVM, sendo comum que essas abordagens coexistam e se complementem dentro de uma mesma solução corporativa.

QUESTÕES COMENTADAS

1. (FGV / TJ RJ - 2024) Com relação à arquitetura de software em camadas, avalie se as afirmativas a seguir são verdadeiras (V) ou falsas (F).

() A arquitetura de software em camadas é uma arquitetura que divide o software em camadas, cada uma com uma responsabilidade específica.

() A arquitetura de software em camadas é uma arquitetura que facilita a manutenção e evolução do software, pois cada camada pode ser desenvolvida e mantida independentemente das outras camadas.

() A arquitetura de software em camadas é uma arquitetura que é sempre a melhor escolha para qualquer tipo de software.

As afirmativas são, respectivamente,

- a) V - F - F.
- b) V - V - F.
- c) F - V - F.
- d) F - V - V.
- e) F - F - V.

Comentários:

(I) Correto. A arquitetura em camadas organiza o sistema em partes com funções específicas, o que favorece separação de responsabilidades.

(II) Correto. Esse modelo ajuda na manutenção e evolução, pois cada camada pode ser tratada com mais independência e organização.

(III) Errado. Nem todo software se beneficia igualmente desse padrão; a escolha da arquitetura depende do contexto e das necessidades do sistema.

Gabarito: Letra B

2. (FGV / ALETO - 2024) A Prefeitura de Nova Esperança planeja criar um sistema de gestão para aumentar a eficiência dos serviços e a interação cidadã. Ele precisa se aproveitar de um sistema

de bancos de dados legado. E as regras fiscais serão projetadas futuramente pelos especialistas e legisladores. Sobre a hipótese apresentada, assinale a afirmativa correta.

- a) Uma arquitetura de três camadas, permitindo integração imediata com o banco de dados e suportar adição futura das regras dos legisladores.
- b) Uma arquitetura de três camadas em que a camada central irá interagir com o sistema de banco de dados legado e a primeira camada irá incorporar as regras fiscais.
- c) Uma arquitetura de duas camadas, sendo a primeira para a nova interface com o usuário e a segunda para interagir com o banco de dados legado.
- d) Uma arquitetura monolítica de microsserviços para compensar a perda de performance para acesso aos bancos de dados legados.
- e) Uma arquitetura Web distribuída de 3 camadas em que a última camada irá disponibilizar a interface com o usuário, a segunda constará as regras de negócio e a primeira camada, a de apresentação, irá interagir com os bancos de dados legados.

Comentários:

- (a) Correto. A arquitetura em três camadas separa apresentação, lógica e dados, facilitando integrar o banco legado agora e incluir depois as regras fiscais na camada de negócio.
- (b) Errado. As regras fiscais não ficam na primeira camada, mas na camada de negócio; por isso, a distribuição proposta não é a mais adequada.
- (c) Errado. Em duas camadas, há menor separação de responsabilidades, o que dificulta evoluções futuras, como a inclusão das regras fiscais.
- (d) Errado. "Arquitetura monolítica de microsserviços" mistura conceitos distintos e não atende bem à necessidade de integração organizada com legado e evolução futura.
- (e) Errado. A descrição das camadas está invertida: a interface com o usuário pertence à apresentação, e o acesso ao banco legado não fica na primeira camada.

Gabarito: Letra A

3. (FGV / DPE RO - 2025) O padrão arquitetural MVC foi proposto na década de 70 e desde então vem sendo amplamente utilizado no mercado. Com relação às afirmativas sobre o padrão, avalie as afirmativas a seguir e assinale (V) para a afirmativa verdadeira e (F) para a falsa.

() **Visão** - são as classes responsáveis pela apresentação da interface gráfica do sistema, incluindo janelas, botões, menus, barras de rolagem. A interface gráfica é formada por objetos de visão e contêineres.

() **Container** - são as classes que tratam e interpretam eventos gerados por dispositivos de entrada, como mouse e teclado. Como resultado de tais eventos, contêineres podem solicitar uma alteração no estado do Modelo ou da Visão. Suponha, por exemplo, uma Calculadora. Quando o usuário clica em um botão +, uma classe Controladora deve capturar esse evento e chamar um método do Modelo.

() **Modelo** - são as classes que armazenam os dados manipulados pela aplicação e que têm a ver com o domínio do sistema em construção. Assim, classes de Modelo não têm qualquer conhecimento ou dependência para classes de Visão e Contêineres. Além de dados, classes de Modelo podem conter métodos que alteram o estado dos objetos de domínio.

As afirmativas são, respectivamente,

- a) V - V - F.
- b) V - F - F.
- c) F - V - F.
- d) F - V - V.
- e) F - F - V.

Comentários:

(F) A descrição mistura conceitos e não caracteriza adequadamente a afirmativa como verdadeira no contexto cobrado, por isso ela não entra entre as válidas.

(F) O item atribui ao contêiner um papel de tratamento de eventos que, na lógica apresentada pela questão, não é o considerado correto.

(V) O modelo representa os dados e regras do domínio, podendo alterar estados sem depender de visão e contêineres.

Gabarito: Letra E

4. (FGV / ALEAM - 2025) Em uma aplicação web para o backoffice da Casa, o modelo MVC (Model-View-Controller) é adotado para separar as preocupações do código, promovendo a manutenibilidade e a testabilidade. A principal responsabilidade do Model em uma aplicação MVC clássica, no contexto de uma requisição web é

- a) gerenciar a interação do usuário e receber as entradas para decidir qual ação será executada.
- b) determinar o formato de apresentação dos dados e exibí-los ao usuário.
- c) tratar exclusivamente a lógica de segurança e criptografia da sessão do usuário.
- d) representar a lógica de negócio, persistência de dados e as regras de domínio, sendo totalmente independente da View e do Controller.
- e) atuar como um gateway de API, roteando as requisições HTTP para os serviços de backend.

Comentários:

- (a) Errado. Essa função é típica do Controller, que recebe entradas e decide o fluxo da aplicação.
- (b) Errado. Quem define a apresentação e exibe os dados ao usuário é a View, não o Model.
- (c) Errado. Segurança e criptografia podem existir na aplicação, mas não são responsabilidade exclusiva do Model.
- (d) Correto. O Model concentra a lógica de negócio, regras de domínio e persistência, mantendo independência da View e do Controller.
- (e) Errado. Rotear requisições HTTP para serviços de backend é papel ligado à infraestrutura ou ao controlador, não ao Model.

Gabarito: Letra D

5. (FGV / CGE SP - 2025) O padrão de arquitetura Model-View-Controller (MVC) é amplamente usado em aplicações web, separando responsabilidades para facilitar a manutenção, a reutilização de código e a testabilidade.

Assinale a opção que indica, no MVC, o componente que é responsável por receber as requisições do usuário, processar a entrada, determinar a lógica de negócio que deve ser executada, interagindo com o Model e, por fim, selecionar a View que deve ser apresentada ao usuário.

- a) View.
- b) Model.
- c) Controller.

- d) Repository.
- e) Service.

Comentários:

- (a) Errado. A View cuida da apresentação dos dados ao usuário, não de receber requisições e coordenar o fluxo da aplicação.
- (b) Errado. O Model representa os dados e a lógica de negócio, mas não faz o papel de entrada e encaminhamento das ações do usuário.
- (c) Correto. O Controller recebe a requisição, trata a entrada, aciona o Model quando necessário e define qual View será exibida.
- (d) Errado. Repository é um padrão ligado ao acesso a dados, não ao controle do fluxo entre entrada do usuário, Model e View.
- (e) Errado. Service costuma concentrar regras de negócio em algumas arquiteturas, mas não é o componente central do padrão MVC.

Gabarito: Letra C

6. (FGV / ALESC - 2024) O MVC (Model-View-Controller) é um padrão de arquitetura de software. Com relação ao Padrão MVC, analise os itens a seguir.

I. No MVC, estão presentes três componentes: Model, que representam o negócio; View, que responde pela interface que será apresentada; e, Controller, que integra os componentes model e view, fazendo com que os models possam ser repassados para as views e vice-versa.

II. A semântica do MVC envolve um model, vários componentes Controller e apenas uma única View.

III. O Model encapsula o estado do aplicativo e a funcionalidade principal do negócio.

Está correto o que se afirma em

- a) I, apenas.
- b) I e II, apenas.
- c) I e III, apenas.
- d) II e III, apenas.
- e) I, II e III apenas.

Comentários:

(I) Correto. O MVC é formado por Model, View e Controller; o Controller faz a mediação entre regra de negócio e interface, permitindo a interação entre esses componentes.

(II) Errado. No MVC, não se limita a arquitetura a vários Controllers e apenas uma única View; essa descrição restringe indevidamente o padrão.

(III) Correto. O Model concentra o estado da aplicação e a lógica central do negócio, sendo a parte responsável pelos dados e comportamento do sistema.

Gabarito: Letra C

7. (FGV / ALEP - 2024) Queremos construir uma aplicação WEB em JAVA em três camadas. Sobre a utilização do padrão de projeto MVC nesta aplicação, assinale a afirmativa correta.

- a) A primeira camada, ou camada de apresentação, e a segunda camada, ou camada de negócios, ambas implementam uma versão do MVC.
- b) A camada de apresentação corresponde a View do MVC além de implementar as classes do Model.
- c) A camada de apresentação corresponde a View do padrão MVC, o Controller é implementado na segunda camada utilizando o Model que é implementado na terceira camada.
- d) A primeira camada corresponde a View do padrão MVC e a terceira camada implementa o Controller e o Model do padrão MVC.
- e) A camada de apresentação corresponde ao frontend e o backend corresponde unicamente a View e o Controller do padrão MVC.

Comentários:

(a) Errado. A arquitetura em três camadas não exige que apresentação e negócios implementem, cada uma, uma versão própria do MVC.

(b) Errado. A camada de apresentação se relaciona com a interface, não sendo a responsável por concentrar também as classes do Model.

(c) Correto. Na divisão proposta, a apresentação atua como View, o Controller fica na segunda camada e o Model na terceira, separando bem as responsabilidades.

(d) Errado. A primeira camada pode representar a View, mas não cabe à terceira reunir Controller e Model ao mesmo tempo.

(e) Errado. No padrão MVC, a View está ligada à apresentação; não faz sentido atribuir ao backend unicamente View e Controller.

Gabarito: Letra C

8. (FGV / INPE - 2024) O padrão MVC (Model-View-Controller) tem como objetivo facilitar a manutenção e a escalabilidade de um software. Com relação ao padrão MVC, analise as afirmativas a seguir.

I. A camada Model gerencia o fluxo de dados entre as outras camadas e as interações com o usuário.

II. A camada View gerencia a apresentação e a interface do usuário.

III. A camada Controller contém as regras de negócio, os dados e a lógica da aplicação.

Está correto o que se afirma em

- a) II, apenas.
- b) III, apenas.
- c) I e II, apenas.
- d) II e III apenas.
- e) I, II e III.

Comentários:

(I) Errado. A afirmativa atribui ao Model funções de controle do fluxo e interação com o usuário, que não correspondem ao papel dessa camada no MVC.

(II) Correto. A View é a camada voltada à apresentação das informações e à interface com o usuário, exibindo os dados de forma adequada.

(III) Errado. A afirmativa concentra no Controller elementos ligados a regras de negócio e dados, o que não corresponde à separação proposta pelo MVC.

Gabarito: Letra A

9. (FGV / EPE - 2024) A arquitetura MVC (Model-View-Controller) sugere a divisão de aplicações em três principais grupos de componentes: Models, Views e Controllers, o que ajuda a atingir a separação de interesses. Em relação aos Controllers, analise as afirmativas a seguir e assinale (V) para a verdadeira e (F) para a falsa.

() Atua como ponto de entrada inicial, sendo responsável por escolher os Models que serão empregados na execução bem como qual View será renderizada.

() Caso a lógica do Controller esteja muito complexa, é recomendável mover lógica de negócios para fora do mesmo, insira-a no modelo de domínio, por exemplo.

() Caso um Controller execute com frequência os mesmos tipos de ações, é uma boa prática mover essas ações comuns para filtros.

As afirmativas são, respectivamente,

- a) V - F - V.
- b) V - V - F.
- c) F - V - F.
- d) F - F - V.
- e) V - V - V.

Comentários:

(V) O Controller atua como intermediário da requisição, coordenando o fluxo da aplicação, selecionando os Models necessários e definindo qual View será apresentada;

(V) Em MVC, lógica de negócios complexa deve ficar fora do Controller, normalmente no modelo de domínio ou em serviços, mantendo o Controller mais enxuto;

(V) Ações recorrentes, como autenticação, autorização e validações comuns, podem ser centralizadas em filtros, favorecendo reutilização e manutenção.

Gabarito: Letra E

10.(FGV / MACAEPREV - 2024) A arquitetura de software com 3 camadas combinada com o padrão de design MVC (Model-View-Controller) é uma abordagem comum para o desenvolvimento de aplicações que promove a separação de preocupações e a modularidade. Na arquitetura de software com 3 camadas combinada com o padrão de design MVC (Model-View-Controller), em relação à responsabilidade de cada camada é correto afirmar que a camada

- a) de visão (View) gerencia a lógica de negócios, a camada de controlador (Controller) realiza a persistência de dados, e a camada de modelo (Model) exibe a interface ao usuário.
- b) de modelo (Model) é responsável pela interação com o banco de dados e lógica de negócios, a camada de visão (View) apresenta os dados ao usuário, e a camada de controlador (Controller) gerencia as interações entre a visão e o modelo.
- c) de controlador (Controller) realiza a persistência de dados, a camada de visão (View) lida com a lógica de negócios e a camada de modelo (Model) exibe a interface ao usuário.
- d) de visão (View) lida com a lógica de negócios, a camada de modelo (Model) realiza a comunicação com o banco de dados, e a camada de controlador (Controller) exibe os dados ao usuário.
- e) modelo (Model) gerencia a apresentação dos dados ao usuário, a camada de visão (View) realiza a lógica de negócios, e a camada de controlador (Controller) faz a persistência de dados.

Comentários:

- (a) Errado. A View não cuida da lógica de negócios, o Controller não faz persistência e o Model não exibe interface; as funções estão trocadas.
- (b) Correto. O Model concentra dados e regras de negócio, a View mostra as informações ao usuário e o Controller faz a mediação entre ambos.
- (c) Errado. Persistência não é papel do Controller, lógica de negócios não fica na View e interface não é responsabilidade do Model.
- (d) Errado. A View não trata lógica de negócios, o Model pode se relacionar com dados, mas o Controller não exibe informações ao usuário.
- (e) Errado. Apresentação é papel da View, lógica de negócios fica no Model e persistência não é função típica do Controller.

Gabarito: Letra B

LISTA DE QUESTÕES

1. (FGV / TJ RJ - 2024) Com relação à arquitetura de software em camadas, avalie se as afirmativas a seguir são verdadeiras (V) ou falsas (F).

() A arquitetura de software em camadas é uma arquitetura que divide o software em camadas, cada uma com uma responsabilidade específica.

() A arquitetura de software em camadas é uma arquitetura que facilita a manutenção e evolução do software, pois cada camada pode ser desenvolvida e mantida independentemente das outras camadas.

() A arquitetura de software em camadas é uma arquitetura que é sempre a melhor escolha para qualquer tipo de software.

As afirmativas são, respectivamente,

- a) V - F - F.
- b) V - V - F.
- c) F - V - F.
- d) F - V - V.
- e) F - F - V.

2. (FGV / ALETO - 2024) A Prefeitura de Nova Esperança planeja criar um sistema de gestão para aumentar a eficiência dos serviços e a interação cidadã. Ele precisa se aproveitar de um sistema de bancos de dados legado. E as regras fiscais serão projetadas futuramente pelos especialistas e legisladores. Sobre a hipótese apresentada, assinale a afirmativa correta.

- a) Uma arquitetura de três camadas, permitindo integração imediata com o banco de dados e suportar adição futura das regras dos legisladores.
- b) Uma arquitetura de três camadas em que a camada central irá interagir com o sistema de banco de dados legado e a primeira camada irá incorporar as regras fiscais.
- c) Uma arquitetura de duas camadas, sendo a primeira para a nova interface com o usuário e a segunda para interagir com o banco de dados legado.
- d) Uma arquitetura monolítica de microserviços para compensar a perda de performance para acesso aos bancos de dados legados.

- e) Uma arquitetura Web distribuída de 3 camadas em que a última camada irá disponibilizar a interface com o usuário, a segunda constará as regras de negócio e a primeira camada, a de apresentação, irá interagir com os bancos de dados legados.

3. (FGV / DPE RO - 2025) O padrão arquitetural MVC foi proposto na década de 70 e desde então vem sendo amplamente utilizado no mercado. Com relação às afirmativas sobre o padrão, avalie as afirmativas a seguir e assinale (V) para a afirmativa verdadeira e (F) para a falsa.

() Visão - são as classes responsáveis pela apresentação da interface gráfica do sistema, incluindo janelas, botões, menus, barras de rolagem. A interface gráfica é formada por objetos de visão e contêineres.

() Container - são as classes que tratam e interpretam eventos gerados por dispositivos de entrada, como mouse e teclado. Como resultado de tais eventos, contêineres podem solicitar uma alteração no estado do Modelo ou da Visão. Suponha, por exemplo, uma Calculadora. Quando o usuário clica em um botão +, uma classe Controladora deve capturar esse evento e chamar um método do Modelo.

() Modelo - são as classes que armazenam os dados manipulados pela aplicação e que têm a ver com o domínio do sistema em construção. Assim, classes de Modelo não têm qualquer conhecimento ou dependência para classes de Visão e Contêineres. Além de dados, classes de Modelo podem conter métodos que alteram o estado dos objetos de domínio.

As afirmativas são, respectivamente,

- a) V - V - F.
- b) V - F - F.
- c) F - V - F.
- d) F - V - V.
- e) F - F - V.

4. (FGV / ALEAM - 2025) Em uma aplicação web para o backoffice da Casa, o modelo MVC (Model-View-Controller) é adotado para separar as preocupações do código, promovendo a manutenibilidade e a testabilidade. A principal responsabilidade do Model em uma aplicação MVC clássica, no contexto de uma requisição web é

- a) gerenciar a interação do usuário e receber as entradas para decidir qual ação será executada.
- b) determinar o formato de apresentação dos dados e exibi-los ao usuário.
- c) tratar exclusivamente a lógica de segurança e criptografia da sessão do usuário.

d) representar a lógica de negócio, persistência de dados e as regras de domínio, sendo totalmente independente da View e do Controller.

e) atuar como um gateway de API, roteando as requisições HTTP para os serviços de backend.

5. (FGV / CGE SP - 2025) O padrão de arquitetura Model-View-Controller (MVC) é amplamente usado em aplicações web, separando responsabilidades para facilitar a manutenção, a reutilização de código e a testabilidade.

Assinale a opção que indica, no MVC, o componente que é responsável por receber as requisições do usuário, processar a entrada, determinar a lógica de negócio que deve ser executada, interagindo com o Model e, por fim, selecionar a View que deve ser apresentada ao usuário.

- a) View.
- b) Model.
- c) Controller.
- d) Repository.
- e) Service.

6. (FGV / ALESC - 2024) O MVC (Model-View-Controller) é um padrão de arquitetura de software. Com relação ao Padrão MVC, analise os itens a seguir.

I. No MVC, estão presentes três componentes: Model, que representam o negócio; View, que responde pela interface que será apresentada; e, Controller, que integra os componentes model e view, fazendo com que os models possam ser repassados para as views e vice-versa.

II. A semântica do MVC envolve um model, vários componentes Controller e apenas uma única View.

III. O Model encapsula o estado do aplicativo e a funcionalidade principal do negócio.

Está correto o que se afirma em

- a) I, apenas.
- b) I e II, apenas.
- c) I e III, apenas.
- d) II e III, apenas.
- e) I, II e III apenas.

7. (FGV / ALEP - 2024) Queremos construir uma aplicação WEB em JAVA em três camadas. Sobre a utilização do padrão de projeto MVC nesta aplicação, assinale a afirmativa correta.

- a) A primeira camada, ou camada de apresentação, e a segunda camada, ou camada de negócios, ambas implementam uma versão do MVC.
- b) A camada de apresentação corresponde a View do MVC além de implementar as classes do Model.
- c) A camada de apresentação corresponde a View do padrão MVC, o Controller é implementado na segunda camada utilizando o Model que é implementado na terceira camada.
- d) A primeira camada corresponde a View do padrão MVC e a terceira camada implementa o Controller e o Model do padrão MVC.
- e) A camada de apresentação corresponde ao frontend e o backend corresponde unicamente a View e o Controller do padrão MVC.

8. (FGV / INPE - 2024) O padrão MVC (Model-View-Controller) tem como objetivo facilitar a manutenção e a escalabilidade de um software. Com relação ao padrão MVC, analise as afirmativas a seguir.

I. A camada Model gerencia o fluxo de dados entre as outras camadas e as interações com o usuário.

II. A camada View gerencia a apresentação e a interface do usuário.

III. A camada Controller contém as regras de negócio, os dados e a lógica da aplicação.

Está correto o que se afirma em

- a) II, apenas.
- b) III, apenas.
- c) I e II, apenas.
- d) II e III apenas.
- e) I, II e III.

9. (FGV / EPE - 2024) A arquitetura MVC (Model-View-Controller) sugere a divisão de aplicações em três principais grupos de componentes: Models, Views e Controllers, o que ajuda a atingir a separação de interesses. Em relação aos Controllers, analise as afirmativas a seguir e assinale (V) para a verdadeira e (F) para a falsa.

() Atua como ponto de entrada inicial, sendo responsável por escolher os Models que serão empregados na execução bem como qual View será renderizada.

() Caso a lógica do Controller esteja muito complexa, é recomendável mover lógica de negócios para fora do mesmo, insira-a no modelo de domínio, por exemplo.

() Caso um Controller execute com frequência os mesmos tipos de ações, é uma boa prática mover essas ações comuns para filtros.

As afirmativas são, respectivamente,

- a) V - F - V.
- b) V - V - F.
- c) F - V - F.
- d) F - F - V.
- e) V - V - V.

10.(FGV / MACAEPREV - 2024) A arquitetura de software com 3 camadas combinada com o padrão de design MVC (Model-View-Controller) é uma abordagem comum para o desenvolvimento de aplicações que promove a separação de preocupações e a modularidade. Na arquitetura de software com 3 camadas combinada com o padrão de design MVC (Model-View-Controller), em relação à responsabilidade de cada camada é correto afirmar que a camada

- a) de visão (View) gerencia a lógica de negócios, a camada de controlador (Controller) realiza a persistência de dados, e a camada de modelo (Model) exibe a interface ao usuário.
- b) de modelo (Model) é responsável pela interação com o banco de dados e lógica de negócios, a camada de visão (View) apresenta os dados ao usuário, e a camada de controlador (Controller) gerencia as interações entre a visão e o modelo.
- c) de controlador (Controller) realiza a persistência de dados, a camada de visão (View) lida com a lógica de negócios e a camada de modelo (Model) exibe a interface ao usuário.
- d) de visão (View) lida com a lógica de negócios, a camada de modelo (Model) realiza a comunicação com o banco de dados, e a camada de controlador (Controller) exibe os dados ao usuário.
- e) modelo (Model) gerencia a apresentação dos dados ao usuário, a camada de visão (View) realiza a lógica de negócios, e a camada de controlador (Controller) faz a persistência de dados.

GABARITO

- | | | | | | |
|----|---------|----|---------|-----|---------|
| 1. | LETRA B | 5. | LETRA C | 9. | LETRA E |
| 2. | LETRA A | 6. | LETRA C | 10. | LETRA B |
| 3. | LETRA E | 7. | LETRA C | | |
| 4. | LETRA D | 8. | LETRA A | | |

ESSA LEI TODO MUNDO CONHECE: PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.



1 Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.



2 Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).



3 Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).



4 Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).



5 Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).



6 Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).



7 Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.



8 O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.